



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

MESTERSÉGES INTELLIGENCIA TANSZÉK

**KCMamba: Kalman-vezérelt szelektív
állapottér-modell
robosztus idősor-előrejelzésre**

Témavezető:

Kristian Fenech Arian

oktató

Szerző:

Kovács András (K2WEMI)

programtervező informatikus BSc

Budapest, 2026

SZAKDOLGOZAT TÉMABEJELENTŐ

Hallgató adatai:

Név: Kovács András

Neptun kód: K2WEMI

Képzési adatok:

Szak: programtervező informatikus, alapképzés (BA/BSc/BProf)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Fenech Kristian Arian

munkahelyének neve, tanszéke: ELTE IK, Mesterséges Intelligencia Tanszék

munkahelyének címe: 1117, Budapest, Pázmány Péter sétány 1/C.

beosztás és iskolai végzettsége: Oktató

A szakdolgozat címe: Kalman-alapú állapotter-modelle fejlesztése idősoros becsléshez

A szakdolgozat témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben szakdolgozat témájának leírását)

Motiváció

Számos alkalmazási terület – járműkövetés, robot szenzorfüzió, szenzorhálózatok és energia/pénzügyi előrejelzés – zajos, nagyméretű idősorok való idejű feldolgozását igényli. A gyakorlatban alkalmazott módszerek gyakran számítási szempontból drágák, érzékenyek a mérési anomáliákra, és nehezen skálázhatók hosszú sorozatokra és magas állapotdimenziókra. A szakdolgozat célja egy Kalman-szűrőn alapuló állapotter-modelle megtervezése és kiértékelése.

Célkitűzések

1. Kalman-alapú modelle megtervezése SSM vagy Mamba modellek alapján.
2. Működés kiértékelése ilyen feltételek mellett.
3. Az új megközelítés empirikus összehasonlítása a standard alapvonalakkal több, eltérő adathalmazon.

Kutatási kérdések

- A bizonytalanságbecslés közvetlenül iterálható-e egy strukturált állapotter-modellebe, és milyen mértékben előnyös ez?
- Hogyan viselkedik a modelle különböző zajszintek, részleges szenzorhibák és kiugró értékek esetén?
- Mennyire általánosítható a megközelítés különböző területeken (követés, szenzorfüzió, pénzügyi/energia idősorok)?

Módszertani megközelítés

A szakdolgozat egy Kalman-családba tartozó, állapotter-becslőt épít.

A fejlesztés a Kalman-szűrőben található bizonytalanságbecslés és predikció-frissítés hurkok integrálására összpontosít a Strukturált Állapotter-modellek központi blokkjába.

Értékelési terv és metrikák

A modelle több adathalmazon tipikus alapértékekkel hasonlítjuk össze. A főbb metrikák közé tartozik az RMSE/MAE (becslési hiba), a negatív log-valószínűség vagy a kalibrációs metrikák (bizonytalanságkezelés). A dedikált ablációs vizsgálatok számszerűsítik, hogy az egyes komponensek hogyan járulnak hozzá a sebesség- és stabilitásnövekedéshez.

Budapest, 2025. 10. 15.

Nyilatkozat

Alulírott **Kovács András**, az Eötvös Loránd Tudományegyetem programtervező informatikus BSc szakos hallgatója kijelentem, hogy a szakdolgozatban foglalt feladatot önállóan oldottam meg, és csak a megadott irodalomban felsorolt forrásokat (könyveket, folyóiratokat, az internetet stb.) használtam fel. Minden olyan részt, amelyet szó szerint, vagy azonos tartalomban, de átfogalmazva más forrásból vettem át, egyértelműen jelöltem, és a forrást pontosan idéztem.

Budapest, 2026. április

.....

Kovács András

hallgató

Kivonat

Ez a szakdolgozat a **KCMamba** (Kalman Core-Attention Mamba) architektúrát mutatja be, amely differenciálható Kalman-szűrési mechanizmust integrál a Mamba szelektív állapottér-modellbe (SSM) a zajrobosztus idősor-előrejelzés érdekében. A javasolt architektúra tanult Q_{net} -, R_{net} - és K_{net} -alhálózatokat alkalmaz, amelyek adaptívan szabályozzák a mérési és folyamat zaj arányát, ezáltal a modell zajmentes adatokon passzív előrejelzőként, zajosított bemeneten aktív szűrőként viselkedik.

Empirikus értékelést végzünk három nyilvános LTSF-adatkészleten (Exchange-Rate, ETTh1, ETTh2), szintetikus zajosított ($\sigma \in \{0, 1, 3, 5\}$) és alacsony adatbőségű ($\text{stride} \in \{1, 16\}$) körülmények között. Kísérleteink szerint a KCMamba MSE-degradációja $\sigma=0$ -ról $\sigma=5$ -re átlagosan +1 528%, míg a Fair Mamba baseline esetén +28 234%, ez közel 18,5×-es robusztusság-előnyt jelent.

A szakdolgozat a teljes szoftverrendszer leírását is tartalmazza: egy aszinkron, eseményvezérelt kereskedési motort hexagonális architektúrában, webalapú irányítópultot, visszatesztelő keretrendszert és élő Binance-kereskedési adaptert.

Abstract

This thesis presents **KCMamba** (Kalman Core-Attention Mamba), a hybrid neural architecture that integrates a differentiable Kalman filtering mechanism into the Mamba selective state-space model (SSM) for noise-robust time series forecasting. The proposed architecture employs learned Q_{net} , R_{net} , and K_{net} subnetworks that adaptively regulate the ratio of process and measurement noise, enabling the model to behave as a passive predictor on clean data and as an active filter under noisy conditions.

We conduct empirical evaluation on three public LTSF benchmarks (Exchange-Rate, ETTh1, ETTh2) under synthetically injected noise ($\sigma \in \{0, 1, 3, 5\}$) and low data-availability regimes (stride $\in \{1, 16\}$). Our results show that KCMamba achieves an average MSE degradation of +1 528% as noise increases from $\sigma=0$ to $\sigma=5$, compared to +28 234% for Fair Mamba, a $18.5\times$ robustness advantage.

The thesis also describes the complete production software system: an asynchronous, event-driven trading engine built in a hexagonal architecture, a web-based dashboard, a backtesting framework, and a live Binance trading adapter.

Tartalomjegyzék

Nyilatkozat	2
Kivonat	3
Abstract	4
1. Bevezetés	4
1.1. Motiváció	4
1.2. Célkitűzések	5
1.3. Hozzájárulások	5
1.4. Dolgozat felépítése	6
2. Irodalmi háttér	7
2.1. Kalman-szűrő és általánosításai	7
2.1.1. Tanult Kalman-szűrők	8
2.2. Állapottér-modellek	8
2.2.1. S4	8
2.2.2. Mamba	8
2.3. Hosszú távú idősor-előrejelzés	9
2.3.1. Problémaformulázás	9
2.3.2. Referenciamodellek	9
2.3.3. Adatkészletek	10
2.4. Zajrobusttusság	10
3. A KCMamba architektúra	12
3.1. Áttekintés	12
3.2. KCMamba blokk	14
3.2.1. Bemeneti projekció és 1D-konvolúció	14
3.2.2. Kalman-paraméter hálózatok	14

3.2.3.	Kalman-gain számítás	15
3.2.4.	Állapotfrissítés párhuzamos scan-nel	15
3.2.5.	Összegzés és reziduál kapcsolat	16
3.3.	KCMamba verem	16
3.4.	Paraméterkomplexitás	19
3.5.	Összehasonlítás a standard Mamba-val	20
3.6.	Rendszerarchitektúra	21
3.7.	Modul- és osztályszerkezet	21
4.	Kísérletek és értékelés	23
4.1.	Kísérleti beállítások	23
4.1.1.	Hardver és szoftver	23
4.1.2.	Adatkészletek	23
4.1.3.	Zajmodell	24
4.1.4.	Modellkonfiguráció és tanítási hiperparaméterek	24
4.2.	Eredmények	25
4.2.1.	Exchange-Rate adatkészlet	25
4.2.2.	ETTh2 adatkészlet	26
4.2.3.	Összefoglaló degradációs statisztika	26
4.3.	A Kalman-gain dinamikája	27
4.4.	Diszkusszió	27
4.5.	Hosszú szekvencia vizsgálat	28
5.	A megvalósított rendszer	31
5.1.	Megvalósítási technológiák	31
5.2.	Komponens-terv	32
5.3.	A webalapú irányítópult	32
5.3.1.	Főoldal, Áttekintő nézet	33
5.3.2.	Kereskedési jelzések	33
5.3.3.	Modell diagnosztika	34
5.3.4.	Kalman-szűrő diagnosztika	34
5.3.5.	Offline tanítás	35
5.3.6.	Benchmark és modell-összehasonlítás	35
5.4.	A visszatesztelő keretrendszer	36
5.5.	Kockázatkezelés	36

5.6.	Élő kereskedés Binance-szal	37
5.7.	A rendszer futtatása	38
5.8.	Felhasználói dokumentáció	38
5.8.1.	A szoftver célja és célközönsége	38
5.8.2.	Rendszerkövetelmények	39
5.8.3.	Telepítési útmutató	39
5.8.4.	Használati útmutató	39
5.8.5.	Hibaelhárítás	41
5.9.	Tesztelés	42
5.9.1.	Tesztelési stratégia	42
5.9.2.	Tesztek futtatása	42
5.9.3.	Tesztesetek és eredmények	43
5.9.4.	Tesztelési korlátok és fejlesztési lehetőségek	43
6.	Összegzés	44
6.1.	Főbb eredmények	44
6.2.	Korlátok	45
6.3.	Jövőbeli irányok	46
6.4.	Zárszó	47
	Irodalomjegyzék	48
	Ábrajegyzék	50

1. fejezet

Bevezetés

Az idősor-előrejelzés a gépi tanulás egyik kiterjedten kutatott részterülete, amelynek gyakorlati jelentőségét a valós adatok szinte kivétel nélkül jelen lévő zajossága adja. Szenzorkiesések, aggregációs torzítások és piaci mikrostruktúra-zaj egyaránt rontják a jel/zaj arányt, amelyre a kurrens modellek érzékenynek bizonyulnak. A hosszú távú idősor-előrejelzés (LTSF) feladata ennél is nehezebb: adott L hosszúságú előzmény alapján kell becsülni a következő H lépést, ahol H nagyságrendileg elérheti vagy meghaladhatja L -et.

1.1. Motiváció

A modern szekvencia-model-családok, a Mamba-alapú szelektív állapotter-modellek [1, 2] és a klasszikus rekurrens hálózatok [3] az adatminőségi problémát implicit általánosítás révén kezelik: a zaj- és jelkomponens szétválasztását kizárólag az adatvezérelt reprezentációtanulásra bízzák. Zajmentes adatokon ez az eljárás kiégítően teljesít, ám a megnövelt zajszintre való általánosítóképesség empirikusan nem garantált.

A Kalman-szűrő [4] ezzel szemben *explicit* statisztikai modellt állít fel a folyamat-zajra (Q) és a mérési zajra (R), és ezek arányából származtat egy K gain együtthatót, amely az előzmény-állapot és az aktuális megfigyelés közötti optimális súlyozást határozza meg. Ennek az induktív torzításnak egy differenciálható neurális hálózatba történő beépítése elméleti szempontból ígéretes megközelítés a zajrobustusság növelésére.

Ezen megfontolásra épül a jelen szakdolgozatban bemutatott **KCMamba** architektúra.

1.2. Célkitűzések

Három konkrét kérdést vizsgálunk:

1. Versenyképes-e a KCMamba tiszta adatokon a standard LTSF modellekkel szemben?
2. Mekkora MSE-degradációt szenvednek el a modellek szintetikus zajosított adatokon ($\sigma \in \{0, 1, 3, 5\}$)?
3. Kevés tréning-ablak ($\text{stride} \in \{1, 16\}$) esetén melyik architektúra generalizál jobban?

1.3. Hozzájárulások

- A **KCMamba** architektúra leírása: tanult Q_{net} , R_{net} , K_{net} alhálózatok differenciálható Kalman-szűrőn belül, párhuzamos prefix-scan GPU-implementációval.
- Mérések arról, hogy a tanult K valóban adaptívan változik a zajszinttel: $\sigma = 0$ -nál $K \approx 0,71$ (passzív átengedés), $\sigma = 5$ -nél $K \approx 0,07$ (erős szűrés).
- Összehasonlítás három adatkészleten (Exchange-Rate, ETTh1, ETTh2): a Fair Mamba átlag +28 234% MSE-degradációt mutat zajjal, a KCMamba +1 528%-ot ($18,5\times$ kisebb romlás).
- Hosszú szekvencia vizsgálat (ETTm1, Exchange; $T \in \{96, 192, 336, 512\}$, 10 epoch, FastAI LR-keresővel): a KCMamba $T = 336$, $\sigma = 1$ konfigurációban **31,7%**-kal jobb MSE-t ér el, mint az LSTM (0,502 vs 0,735). $T = 512$, $\sigma = 3$ -nál a KCMamba 0,773, a Fair Mamba 2,121 — a KCMamba 63,6%-kal jobb. $\sigma = 3$ -nál az ETTm1-en a KCMamba MSE monoton csökken $T = 96$ -tól $T = 512$ -ig ($0,673 \rightarrow 0,466$, -31%).
- Nyílt forráskódú implementáció (Python/PyTorch/CUDA) és egy aszinkron kereskedési rendszer a modell deployálásához.

1.4. Dolgozat felépítése

A 2. fejezet összefoglalja a Kalman-szűrő, az SSM-ek és az LTSF-kutatás releváns irodalmát. A 3. fejezet ismerteti a KCMamba architektúrát és matematikai háttérét. A 4. fejezetben a kísérleti beállítás és az eredmények szerepelnek. A 5. fejezet a megvalósított szoftverrendszert tárgyalja. Az 6. fejezet levonja a következtetéseket.

2. fejezet

Irodalmi háttér

2.1. Kalman-szűrő és általánosításai

A Kalman-szűrő [4] lineáris dinamikus rendszerek optimális Bayes-becslője, ha additív Gauss-zajt feltételezünk. Az állapotfrissítés:

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(y_t - H\hat{x}_{t|t-1}), \quad (2.1)$$

ahol $\hat{x}_{t|t} \in \mathbb{R}^n$ az a posteriori állapotbecslés (t időpont összes mérése alapján), $\hat{x}_{t|t-1}$ az a priori becslés (az előző állapotból prediktálva), $y_t \in \mathbb{R}^m$ a t -edik mérés, $H \in \mathbb{R}^{m \times n}$ a mérési mátrix (állapotból megfigyelést számít), és $(y_t - H\hat{x}_{t|t-1})$ az *innováció*: a tényleges és a becsült mérés különbsége. A K_t Kalman-gain mátrix értékét a predikciós és mérési kovariancia aránya adja meg:

$$K_t = P_{t|t-1}H^\top (HP_{t|t-1}H^\top + R)^{-1}, \quad (2.2)$$

ahol $P_{t|t-1} \in \mathbb{R}^{n \times n}$ az a priori állapot-kovariancia (a becslés bizonytalansága), $R \in \mathbb{R}^{m \times m}$ a mérési zaj kovarianciája, és $H^\top$ a mérési mátrix transzponáltja. Ha $R \rightarrow \infty$, akkor $K \rightarrow 0$: a szűrő a mérésnek alacsony súlyt tulajdonít és az előzmény-állaputra támaszkodik. Ha $R \rightarrow 0$, akkor $K \rightarrow 1$: a mérés döntő súlyt kap az állapotfrissítésben. Ez az arány-alapú dinamikus súlyozás a KCMamba mechanizmusának is kiindulópontja.

2.1.1. Tanult Kalman-szűrők

A neurális Kalman-irodalomban a F, H, Q, R mátrixokat hálózatokkal paraméterezik [5, 6]. A *Deep Kalman Filter* [5] nemlineáris átmenetet enged meg; a *Kalman Variational Autoencoder* [6] a latens állapotok szétválasztott reprezentációját tanítja. A KCMamba ezeken az elveken épít, de nem autoencoderként, hanem szekvencia-előrejelzőként.

2.2. Állapottér-modellek

2.2.1. S4

Az S4 (Structured State Spaces for Sequences) [1] egy folytonos-idejű állapotter-modellből indul ki, amelyben egy d -dimenziós bemeneti sorozatot egy N -dimenziós látens dinamikán keresztül modellez:

$$h'(t) = Ah(t) + Bu(t), \quad (2.3)$$

$$y(t) = Ch(t), \quad (2.4)$$

ahol $A \in \mathbb{R}^{N \times N}$ az állapotátmeneti mátrix, $B \in \mathbb{R}^{N \times d}$ a bemeneti projekció, $C \in \mathbb{R}^{d \times N}$ a kimeneti projekció, és $h(t) \in \mathbb{R}^N$ a látens állapot. Diszkrétizálva egy Δ időlépéssel:

$$h_t = \bar{A} h_{t-1} + \bar{B} u_t, \quad y_t = C h_t, \quad (2.5)$$

ahol $\bar{A} = e^{\Delta A}$ és $\bar{B} \approx (\Delta A)^{-1}(e^{\Delta A} - I)\Delta B$. Az S4 kulcsinnovációja az A mátrix HiPPO-inicializációja, amely az állapotnak polinomiális bázisú hosszú távú memóriát biztosít. A konvolúciós nézet, $\bar{K} = (C\bar{B}, C\bar{A}\bar{B}, \dots, C\bar{A}^{T-1}\bar{B})$ kernel, lehetővé teszi, hogy a tanítás $O(T \log T)$ -ben FFT-vel végezhető, míg az inferencia a rekurrens formában $O(1)$ memóriával fut lépésenként.

2.2.2. Mamba

A Mamba [2] az S4 alapjaira építve bevezeti az *input-függő szelektivitást*: míg az S4-ben az A, B, C mátrixok rögzítettek a teljes szekvenciára, a Mamba ezeket az aktuális bemenettől függővé teszi. Ez a szelektivitás a lényege: a modell minden időlépésben eldönti, mit őrizzen meg az állapotban és mit felejtson el. A bemeneti projekciós mátrix $B(u_t) \in \mathbb{R}^{N \times d}$, a kimeneti projekció $C(u_t) \in \mathbb{R}^{d \times N}$ és az időlépté-

kező $\Delta(u_t) \in \mathbb{R}^d$ mindegyike az aktuális $u_t \in \mathbb{R}^d$ bemeneti vektortól függ:

$$h_t = \overline{A}(u_t) h_{t-1} + \overline{B}(u_t) u_t, \quad y_t = C(u_t) h_t, \quad (2.6)$$

ahol $h_t \in \mathbb{R}^N$ a látens állapotvektor (N az állapotméret), $\overline{A}(u_t) = e^{\Delta(u_t)A}$ és $\overline{B}(u_t) \approx \Delta(u_t)B$ a Δ alapján diszkretizált mátrixok. A modell így szelektíven dönti el, hogy a bemeneti sorozat melyik részét tárolja tartósan az állapotban, és melyiket hagyja figyelmen kívül. Hardver-barát párhuzamos prefix-scan implementációval $O(T)$ memória- és időkomplexitást ér el.

Kísérleteinkben a Mamba egy csatornánkénti (*channel-wise*) változatát alkalmazzuk összehasonlítási alapként: minden bemeneti változót önállóan, azonos N -dimenziós állapottérben dolgoz fel, a KCMamba-éval azonos hiperparaméterekkel és tanítási protokoll szerint. A konfiguráció közös tartásával az eredménykülönbség kizárólag az architektúrális eltérésnek tulajdonítható, a paraméterszámnak vagy az adatmennyiségnek nem. Az irodalom ezt a beállítást *Fair Mamba* néven hivatkozza.

2.3. Hosszú távú idősor-előrejelzés

2.3.1. Problémaformulázás

Adott $\mathbf{X} = (x_1, \dots, x_L) \in \mathbb{R}^{L \times d}$; a cél $\hat{\mathbf{X}} = (\hat{x}_{L+1}, \dots, \hat{x}_{L+H})$ előrejelzése nMSE minimalizálásával.

2.3.2. Referenciamodellek

LSTM

A Long Short-Term Memory [3] hálózat a rekurrens modellek közül a legszélesebb körben alkalmazott idősor-előrejelző architektúra. A *forget gate* $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ szabályozza, hogy az előző cellaállapot mekkora hányada maradjon meg; az *input gate* $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ határozza meg az új információ beáramlásának mértékét. Ez a kapuzási mechanizmus adaptív szűrést valósít meg, ami funkcionálisan rokon a Kalman-gain szerepével, azzal a különbséggel, hogy az LSTM-ben nincs explicit zajmodell. Kísérleteinkben az LSTM erős baseline, különösen sok adat és magas zaj esetén.

ARIMA

Az $\text{ARIMA}(p, d, q)$ modell a Box–Jenkins-eljárás klasszikus statisztikai módszerére [7]. A d -szeres differenciálás stacionerizálja az idősort, az $\text{AR}(p)$ tag lineáris autoregressziót, az $\text{MA}(q)$ tag mozgóátlagot modellez. Az ARIMA zajtalan adatokon rendkívül hatékony, és a legkisebb MSE-t éri el $\sigma = 0$ mellett; ugyanakkor $\sigma \geq 3$ felett katasztrofálisan összeomlik, mert a lineáris struktúra nem képes szétválasztani a jelet és a zajt.

2.3.3. Adatkészletek

- **Exchange-Rate** [8]: 8 valutaárfolyam, 7 588 napi minta. Majdnem egységgyök-közelítő dinamikája miatt a Kalman-struktúra különösen hatékony rajta.
- **ETTh1 / ETTh2** [9]: Transzformátor-állomás óránkénti hőmérsékleti adatai ($d = 7$, 17 420 lépés). Az ETTh1 erősen periodikus, az ETTh2 zajosabb.
- **Weather**: 21 meteorológiai változó (hőmérséklet, páratartalom, légnyomás stb.) 10 perces mintavételezéssel, összesen 52 696 lépés. A magas dimenzió és a változók közötti erős korreláció tesztel a modellek többváltozós kezelőképességét.
- **ECL** (Electricity Consuming Load): 321 ügyfél óránkénti villamosenergia-fogyasztása, 26 304 lépés. A sok csatorna és az erős napi/heti periodikusság miatt gyakori LTSF-benchmark.

2.4. Zajrobusttusság

Az idősor-modellek zajérzékenységének szisztematikus elemzése az LTSF-irodalomban aláreprézentált terület. A PatchTST [10] patch-tokenizálással csökkenti a szomszédos mintavételezési torzítást; ez közvetett zajcsökkentő hatással jár, bár nem ez az eljárás fő célkitűzése. A zajos adatokkal augmentált tanításon alapuló robusttusság-növelés az LTSF-kutatásban eddig kevés figyelmet kapott.

A jelen munkában az értékelési protokoll: $\tilde{x}_t = x_t + \varepsilon_t$, ahol $x_t \in \mathbb{R}^d$ az eredeti idősorjel, $\varepsilon_t \sim \mathcal{N}(0, \sigma^2 I)$ additív Gauss-zaj ($\sigma > 0$ a szórás, $I \in \mathbb{R}^{d \times d}$ az egységmátrix), és $\sigma \in \{0, 1, 3, 5\}$ a vizsgált zajszintek halmaza. A tanító- és teszthalmazt egyaránt

azonos σ -val zajosítjuk, tehát nem domain-shift-ről, hanem egyenletes adatminőség-romlásról van szó.

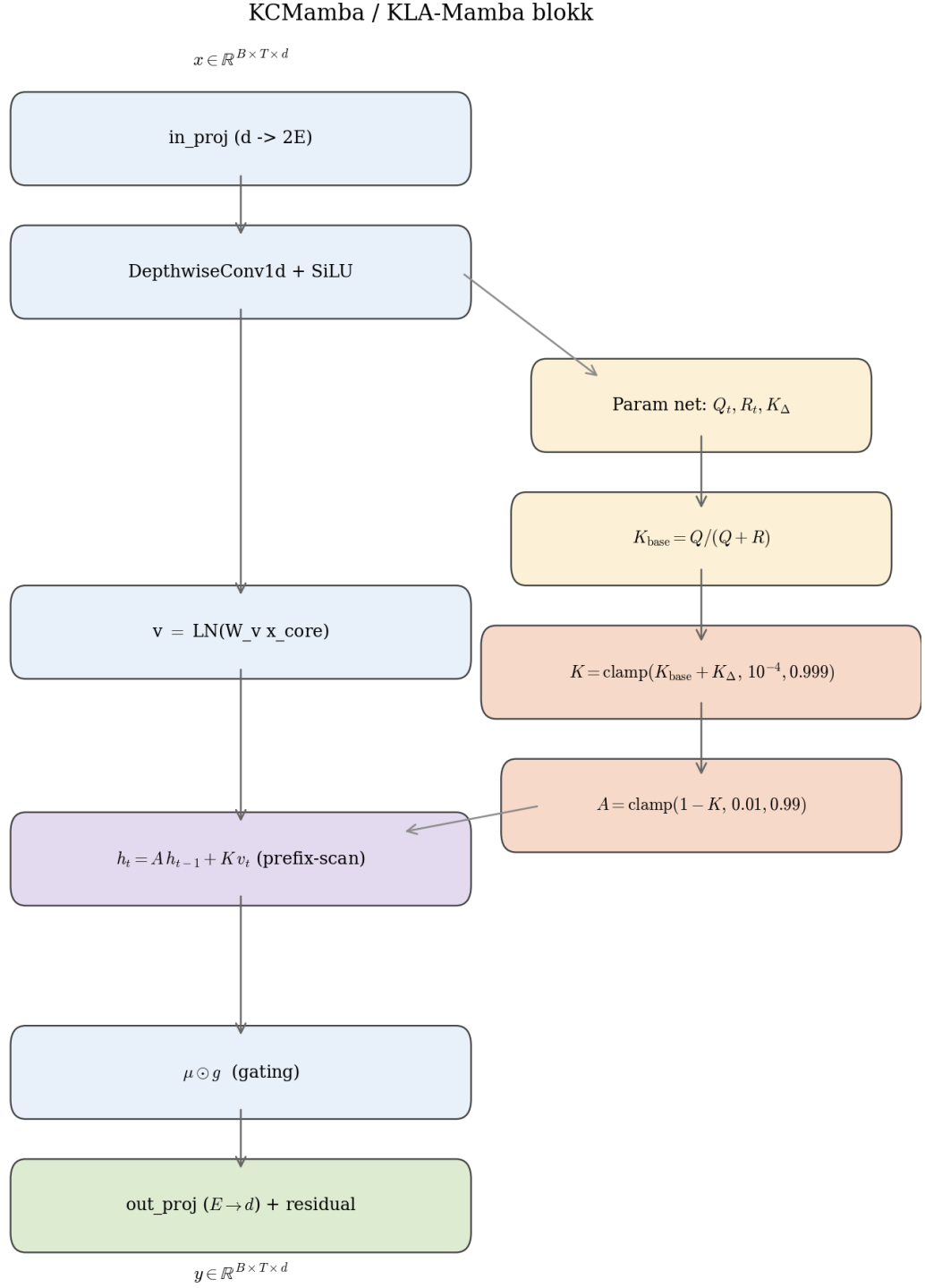
3. fejezet

A KCMamba architektúra

3.1. Áttekintés

A KCMamba (Kalman Core-Attention Mamba) egy hibrid neurális architektúra, amely a Mamba SSM [2] rekurráns dinamikájába egy differenciálható, tartalomfüggő Kalman-szűrő réteget integrál. A modell nem rögzített A felejtési faktorral dolgozik, hanem egy, a bemenetből tanult K -gainből származtatott $A = 1 - K$ együtthatót használ az állapotfrissítésben. Ez lehetővé teszi, hogy az előzmény- állapot és az aktuális megfigyelés közötti súlyozás a zajszintnek megfelelően, adaptívan álljon be.

A modell két szintből épül fel: az alap feldolgozó egység a **KCMambaBlock** (tartalom-függő Kalman-szűrés és állapotfrissítés), fölötte a **KCMambaStack**: N egymás utáni KC-blokk cross-attention réteggel a végén. A cross-attention réteg szerepe, hogy ellensúlyozza a Mamba SSM rekurráns dinamikájából adódó, az utolsó időlépések irányába mutató reprezentációs torzítást; így a végső predikcióba a lassabb időskálájú (hosszú távú) trendek is bevonhatók.



3.1. ábra. A KCMamba blokk teljes adatfolyam-diagramja. A sárga blokkok (Q_{net} , R_{net} , K_{net}) tanítják a Kalman-statisztikákat; a lila blokkban történik a párhuzamos prefix-scan GPU-n $O(\log T)$ szinkronizációval; a zöld blokkok a bemeneti/kimeneti projekciókat jelölik.

3.2. KCMamba blokk

Legyen a bemenő szekvencia $x \in \mathbb{R}^{B \times T \times d}$ (B : kötegméret, T : szekvenciahossz, d : jellemzők száma).

3.2.1. Bemeneti projekció és 1D-konvolúció

A bemenetet két részre vetítjük egy lineáris réteggel, majd SiLU aktivációs kapuzást alkalmazunk:

$$[x_{\text{cw}}, x_{\text{gate}}] = W_{\text{in}} x, \quad (3.1)$$

$$\text{gate} = \text{SiLU}(x_{\text{gate}}), \quad (3.2)$$

ahol $W_{\text{in}} \in \mathbb{R}^{2d_{\text{inner}} \times d}$ a bemeneti projekciós mátrix (d_{inner} a belső csatornadimenzió), x_{cw} a Kalman-szűrési ághoz, x_{gate} a kapuzáshoz továbbított csatorna, és $\text{SiLU}(z) = z \cdot \sigma(z)$ a Sigmoid Linear Unit aktiváció ($\sigma(z) = 1/(1 + e^{-z})$ a logisztikus szigmoid). Az x_{cw} csatornán egy depthwise 1D-konvolúció ($k = 4$ kernel) biztosítja a lokális kontextus-integrációt:

$$x_{\text{core}} = \text{SiLU}(\text{Conv1D}(x_{\text{cw}})). \quad (3.3)$$

3.2.2. Kalman-paraméter hálózatok

Három kis MLP-alhálózat tanítja meg a Kalman-szűrő statisztikáit:

$$Q_t = \text{softplus}(Q_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(q_{\text{scale}}), \quad (3.4)$$

$$R_t = \text{softplus}(R_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(r_{\text{scale}}). \quad (3.5)$$

ahol $Q_t > 0$ a *folymat zaj* becsült varianciája (az állapot- dinamikára vonatkozó bizonytalanság), $R_t > 0$ a *mérési zaj* becsült varianciája (a bemeneti megfigyelés bizonytalansága), $q_{\text{scale}}, r_{\text{scale}} \in \mathbb{R}$ tanítható skalár paraméterek, és $\text{softplus}(z) = \ln(1 + e^z) > 0$ garantálja a pozitivitást. A $q_{\text{scale}} = -2,0$, $r_{\text{scale}} = 1,5$ inicializációk biztosítják, hogy kezdetben $R \gg Q$, vagyis $K_{\text{base}} \approx 0.07$ (erős szűrés alap eset).

3.2.3. Kalman-gain számítás

Az alap Kalman-gain a folyamat- és mérési zaj *arányából* következik:

$$K_{\text{base},t} = \frac{Q_t}{Q_t + R_t + \varepsilon}, \quad (3.6)$$

ahol $\varepsilon = 10^{-8}$ egy kis numerikus stabilitási konstans, amelynek célja megakadályozni a nullával való osztást. Ehhez járul egy tartalom-függő korrekció, amelyet a K_{net} alhálózat biztosít:

$$K_t = K_{\text{base},t} + 0.3(\sigma_{\text{sig}}(K_{\text{net}}(x_{\text{core}})) - 0.5), \quad (3.7)$$

ahol $\sigma_{\text{sig}}(\cdot) = 1/(1 + e^{-(\cdot)})$ a logisztikus szigmoid függvény (értéke $(0, 1)$ -ben van), a -0.5 eltolás nullára centrálja a korrekciót, és a 0.3 szorzó $(-0.15, +0.15)$ -re korlátozza hatókörét. A végeredmény numerikusan stabil marad a fizikailag érvelmes tartományban, ezért szoftveresen (a felhasználói kódban) nem kell külön beavatkozni.

3.2.4. Állapotfrissítés párhuzamos scan-nel

Az $A_t = 1 - K_t$ felejtési faktoral a diszkrét Kalman-rekurzió:

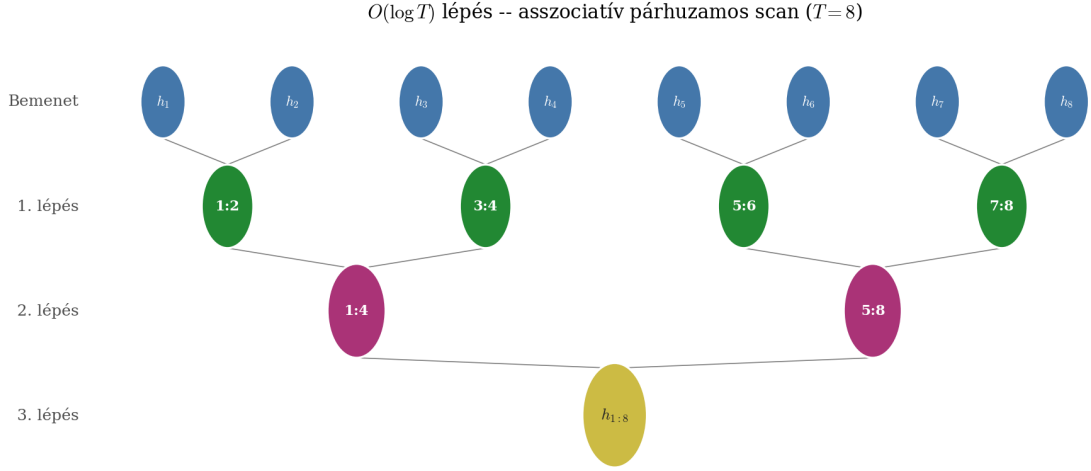
$$\begin{aligned} h_t &= A_t \odot h_{t-1} + K_t \odot v_t, \\ v_t &= \text{LayerNorm}(W_v x_{\text{core},t}), \end{aligned} \quad (3.8)$$

ahol $A_t \in (0, 1)^{d_{\text{inner}}}$ a felejtési faktor vektora ($A_t \rightarrow 1$: erős memória, $A_t \rightarrow 0$: erős frissítés), $\odot$ az elem-szintű (Hadamard) szorzás, $h_{t-1} \in \mathbb{R}^{d_{\text{inner}}}$ az előző lépés látens állapota, v_t az aktuális értékvektor, és $W_v \in \mathbb{R}^{d_{\text{inner}} \times d_{\text{inner}}}$ tanítható projekció. A rekurzió hatékony kiértékelése a Blelloch-féle párhuzamos prefix-scan algoritmus-sal [11] valósul meg:

$$A_{\text{new}}[t] \leftarrow A[t] \cdot A[t - \text{step}], \quad (3.9)$$

$$B_{\text{new}}[t] \leftarrow B[t] + A_{\text{orig}}[t] \cdot B[t - \text{step}], \quad (3.10)$$

ahol $A[t]$ és $B[t]$ a prefixek asszociatív páros eleme $((a_t, b_t))$, ahol a_t a halmozódó felejtési faktor, b_t a halmozódó bemenet-összeg; ezek *nem* azonosak az SSM A , B mátrixaival), és a lépésköz $\text{step} \in \{1, 2, 4, \dots, T/2\}$ duplázódik. A rekurzió $O(\log T)$ szinkronizációs lépéssel párhuzamosítható GPU-n.



3.2. ábra. A Blelloch-féle párhuzamos prefix-scan vizualizációja 8 elemű szekvencián. Minden lépésben a $step$ -nyi távolságú két elem kombinálódik; $\log_2 T = 3$ lépés után minden h_t tartalmazza az előzmény-összesítést. Ez teszi lehetővé a szekvenciális állapotfrissítés hatékony GPU-s párhuzamosítását.

3.2.5. Összegzés és reziduál kapcsolat

A Kalman-állapot-sorozat és a kapuzás kombinálja a kimenetet:

$$y_{KC} = W_{\text{out}}(\mu \odot \text{gate}), \quad (3.11)$$

ahol $\mu = (h_1, \dots, h_T) \in \mathbb{R}^{T \times d_{\text{inner}}}$ az összes időlépés Kalman-állapota, $W_{\text{out}} \in \mathbb{R}^{d \times d_{\text{inner}}}$ a kimeneti projekció, és $\odot$ az elem-szintű szorzás. Egy skalárkapuzással vezérelt reziduál kapcsolat biztosítja a gradiens átmenő útját:

$$\text{out} = y_{KC} + \sigma(b_{\text{res}}) \cdot (W_{\text{res}} x - y_{KC}), \quad (3.12)$$

ahol $b_{\text{res}} \in \mathbb{R}$ tanítható skalár ($b_{\text{res}} = -1,0$ inicializációval $\sigma(b_{\text{res}}) \approx 0,27$), W_{res} a reziduál projekció, és a kis $\sigma(b_{\text{res}})$ alaphelyzetben az SSM-ágot részesíti előnyben.

3.3. KCMamba verem

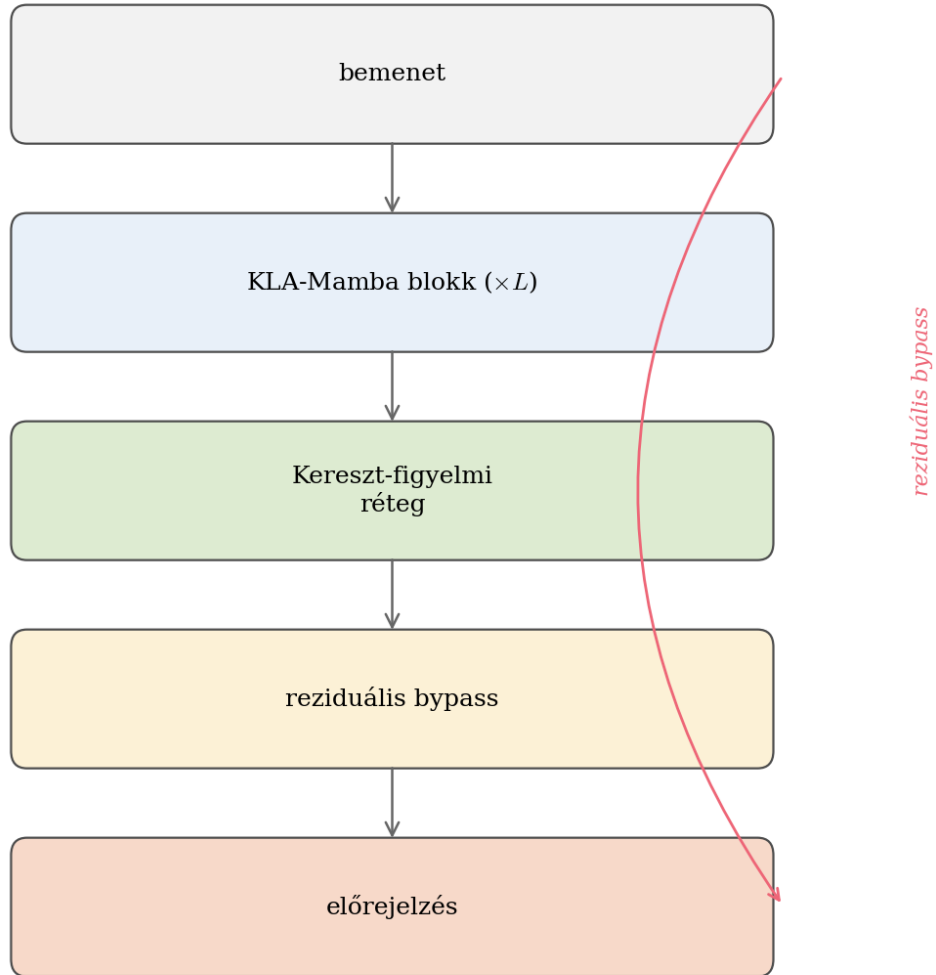
A verem N rétegből áll; köztük LayerNorm rétegek (pre-norm stílus) akadályozzák az aktivációs skála sodródását.

A verem végére egy *multi-scale cross-attention* réteget illesztünk. Ennek oka, hogy a Mamba rekurrens dinamikája a rejtett állapotba a közelmúlt lépéseit súlyozottan erősebben kódolja, mint a régmúltat; egy hosszú-horizontú előrejelzéshez

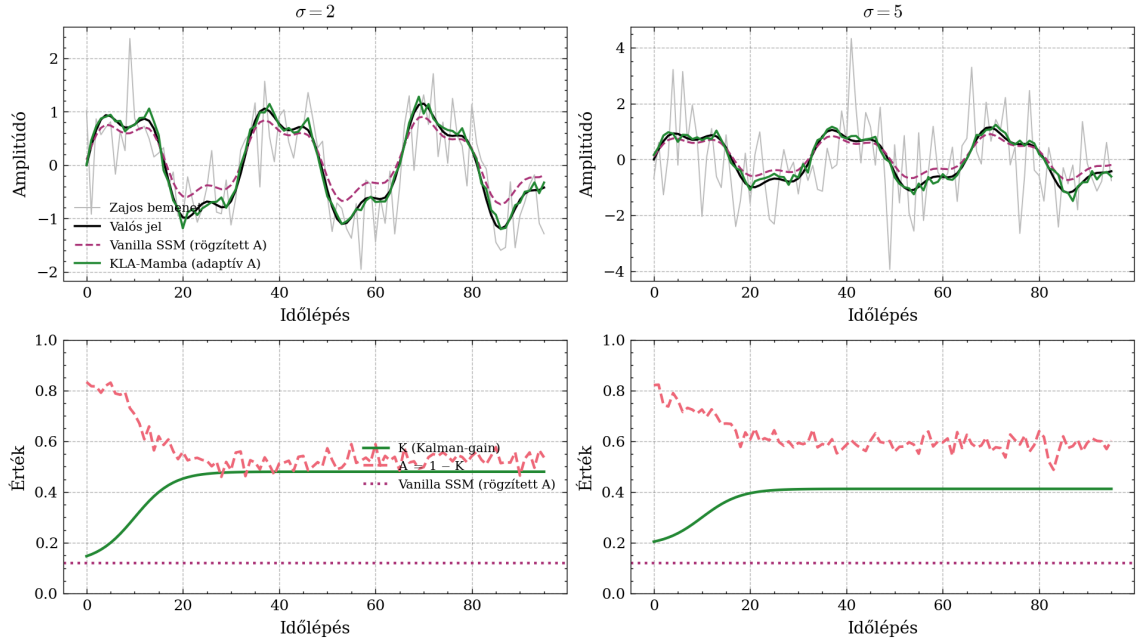
azonban a lassú trendeket is hozzá kell férni. A cross-attention a jelenlegi állapotot (lekérdezést) összeveti a szekvencia s -edik almintá (key/value) halmazával, így rövid és hosszú időskálákat egyaránt számításba vesz:

$$\text{out}[T] = h[T] + \text{MHA}(h[T], h[::s], h[::s]), \quad (3.13)$$

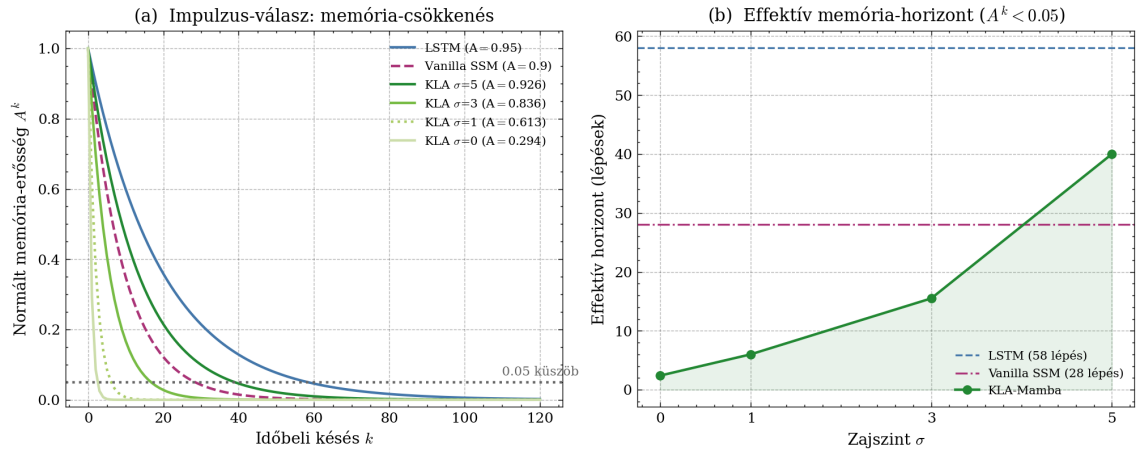
ahol $h[T] \in \mathbb{R}^{1 \times d}$ a legutolsó időlépés reprezentációja (lekérdezés), $h[::s] \in \mathbb{R}^{\lfloor T/s \rfloor \times d}$ az s -edik minden pozícióból vett alámintavétel (kulcs és érték, lassú időskála), és $\text{MHA}(\text{lekérdezés}, \text{kulcs}, \text{érték})$ a multi-head cross-attention a megszokott értelemben [12].



3.3. ábra. A KCMamba verem (stack) felépítése. Bemenettől kimenetig: input projekció, 2 KC-blokk LayerNorm rétegekkel, output normalizáció, majd cross-attention réteg a lassú ($s = 4$) időskála integrálásához.



3.4. ábra. Szintetikus jel szűrése: **vanilla SSM** (standard Mamba, rögzített A) vs. **KCMamba** (adaptív K -gain) közepes ($\sigma = 2$) és erős ($\sigma = 5$) zajnál. *Felső sorok:* az előrejelzett jel a valódihoz képest; a KCMamba szorosabban követ zajban is. *Alsó sorok:* a tanult K -értékek időbeli változása. A vanilla SSM A -ja rögzített, a KCMamba A -ja adaptívan nő zajjal.



3.5. ábra. **Bal:** Impulzus-válasz függvény: az állapot exponenciális csökkenése $h_k = A^k$ a késő k függvényében. A KCMamba A -értéke adaptívan nő a zajszinttel ($0.294 \rightarrow 0.926$), ezzel a memória-horizont is növekszik. **Jobb:** Effektív memória-horizont (az a lépésszám, ameddig $A^k > 5\%$) a zajszint függvényében. Az LSTM és vanilla SSM rögzített horizonttal rendelkeznek, a KCMamba zajban automatikusan hosszabb memóriát tart fenn.

3.4. Paraméterkomplexitás

Modell	Paraméterek (k)	Rétegek	Állapotméret
LSTM	71,7	1	128
Fair Mamba	53,2	2	H=4, D=32
KCMamba	46,1	2	H=4, D=32

3.1. táblázat. A kísérletekben használt modellek paraméterkomplexitása (medium méretkonfiguráció). A KCMamba kevesebb paraméterrel rendelkezik, mint a Fair Mamba, mivel a $Q_{\text{net}}/R_{\text{net}}/K_{\text{net}}$ alhálózatok az SSM-blokkokból az erőforrásokat átirányítják.

3.5. Összehasonlítás a standard Mamba-val

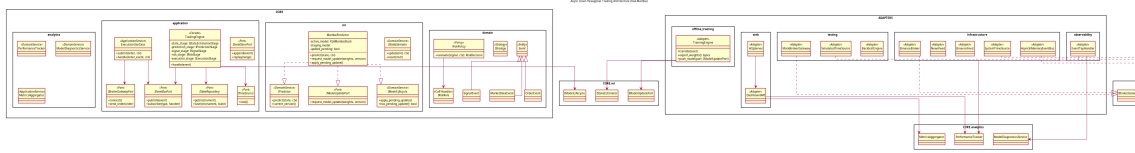
A legfontosabb architektúráis különbség a standard (vanilla) SSM és a KCMamba között az A felejtési faktor kezelésében keresendő. A standard Mamba [2] az A mátrixot a Δ időléptékű paraméteren keresztül tanulja, de ez a reprezentáció nem különíti el a jel- és zajkomponenst: Δ egyszerre jelöli ki az adatstruktúra tanulási dinamikáját és a zajszűrést.

A KCMamba ezt a kettősséget bontja meg egy explicit Q/R zajtartomány-modell bevezetésével. A $K = Q/(Q + R)$ arány határozza meg, hogy az állapotfrissítés az aktuális mérésre vagy az előzmény-állapotra támaszkodik-e erősebben. Magas zaj mellett ($\sigma = 5$) $K \rightarrow 0.073$, azaz $A = 0.927$: a modell erős memóriát tart fenn, és kiszűri a zajos méréseket (erős szűrési mód). Alacsony zaj mellett ($\sigma = 0$) $K = 0.706$, és a modell az aktuális megfigyelést követi (*innovációs mód*).

Ezt a szintetikus teszten a 3.4. ábra demonstrálja: a standard SSM rögzített $A = 0.88$ értékkel dolgozik mindkét zajszinten, míg a KCMamba A -ja a zajszinttel arányosan módosul (3.5. ábra).

3.6. Rendszerarchitektúra

A KCMamba egy hexagonálisan strukturált kereskedési rendszerbe épül be. A 3.6. ábra a főbb komponenseket mutatja:



3.6. ábra. A KCMamba alapú kereskedési rendszer hexagonális architektúrájának áttekintése. A **Core** csomag tartalmazza a TradingEngine-t (pipeline: állapotbecslés → predikció → szignál → kockázat → végrehajtás), a domain modelleket és az alkalmazásportokat. A külső adapterek (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) kizárólag az interfészeken (IEventBusPort, IBrokerGatewayPort stb.) keresztül kommunikálnak a Core-ral.

3.7. Modul- és osztályszerkezet

A forráskód négy fő csomagra oszlik. A `core/` nem hivatkozik egyetlen adapterre sem; az `adapters/` kód kizárólag a `ports.py` interfészeken keresztül éri el.

Csomag	Modul / osztály	Felelősség
core/domain/	models.py strategy.py risk.py events.py	Adattípusok: Candle, Position, Order, Fill ThresholdStrategy: Buy/Sell jelzéslogika, min_edge, max_sigma szűrők RiskPolicy: kockázati szabálykészlet (MaxQty, Stop-loss, napi veszteség-limit) Domain-események: PriceEvent, SignalEvent, FillEvent
core/application/	engine.py execution.py ports.py stages.py stores.py	TradingEngine: fő pipeline-orkesztráló; belső állapotgép a kereskedési ciklushoz Megbízás-összeállítás, mérekszámítás és IBrokerGatewayPort-ra küldés Absztrakt interfészek: IEventBusPort, IBrokerGatewayPort, IMarketFeedPort, ITimeSourcePort Pipeline lépések: PredictionStage, SignalStage, RiskStage, ExecutionStage In-memory állapot: PositionStore, SignalStore, EquityStore
core/ml/	kc_mamba.py mamba_kalman.py services.py feature_engineering.py	KCMambaModel: PyTorch modul, forward(x) → előrejelzés KCMambaBlock: Kalman-szűrő réteg, párhuzamos prefix-scan MLService: modellbetöltés, online frissítés, előrejelzés-gyorsítótár OHLCV → jellemzővektor transzformáció, normalizáció, csúszóablak
adapters/	binance_broker.py binance_feed.py event_bus.py broker.py (testing) dashboard.py (web)	BinanceBroker: élő megbízás-végrehajtás REST API-n BinanceFeed: WebSocket árfolyamstream, reconnect logika InMemoryEventBus: aszinkron, asyncio-alapú eseménytovábbítás PaperBroker, SimulationWallet: szimulált végrehajtás visszateszteléshez DashboardServer: aiohttp HTTP/WS szerver, REST JSON végpontok

3.2. táblázat. A rendszer modul- és osztályszerkezete. Minden osztály kizárólag a saját csomagjának adattípusaira és interfészeire támaszkodik; a keresztírányú függőségeket a ports.py absztrakciói közvetítik.

4. fejezet

Kísérletek és értékelés

4.1. Kísérleti beállítások

4.1.1. Hardver és szoftver

A futtatási környezet: NVIDIA GeForce RTX 3060 (12 GB VRAM), PyTorch 2.x, CUDA 12, float32 pontosság.

4.1.2. Adatkészletek

A fő kérdés három adatkészlet körnéhezódött (részletes leírás a 4.1.2. alfejezetben):

- **Exchange-Rate**, 8 valutaárfolyam, 1990–2016 (7 588 nap, 8 változó) [8].
- **ETTh1**, kínai transzformátorállomás, óránkénti terhelés és hőmérséklet (17 420 lépés, 7 változó) [9].
- **ETTh2**, ugyanaz az állomás, másik csatorna-sorozat (17 420 lépés, 7 változó) [9].

A **Weather** (21 meteorológiai változó, 52 696 lépés) és az **ECL** (321 csatorna, 26 304 lépés) benchmarkok a nagyobb dimenzióság és a hosszú periodicitás vizsgálatára következő mérési lépésben skálázandók; a jelen dolgozat keretein belül csak a három fenti adatkészleten futó eredmények szerepelnek, mert a tanulási futamok száma ($4 \text{ zajszint} \times 2 \text{ stride} \times 4 \text{ modell} \times 3 \text{ adat} = 96 \text{ futam}$) már így is nagyobb, mint amit egyetlen RTX 3060 hardverrel ésszerű időn belül elvégeztem.

4.1.3. Zajmodell

A zajosítás analitikusan:

$$\hat{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I), \quad \sigma \in \{0, 1, 3, 5\}. \quad (4.1)$$

A tanító- és tesztthalmazt azonos σ -val zajosítottam, így a vizsgált probléma nem domain-shift, hanem egységes adatminőség-romlás.

4.1.4. Modellkonfiguráció és tanítási hiperparaméterek

Hiperparaméter	LSTM	Fair Mamba	KCMamba
Epoch	5	5	5
Optimalizáló	Adam	Adam	Adam
Tanulási sebesség	3e-3	3e-3	3e-3
Kötegméret	32	32	32
Szekvenciahossz	96	96	96
Előrejelzési horizont	96	96	96
Rejtett dim (d)	128	128	128
Fejek (H)	n.a.	4	4
Állapotméret (D)	n.a.	32	32
Rétegek	1	2	2
Kernel	n.a.	4	4
Paraméterek	71,0k	53,2k	46,1k

4.1. táblázat. Modell-konfigurációk és tanítási hiperparaméterek a zajrobusztussági kísérletekhez. A KCMamba kevesebb paraméterrel rendelkezik, mint a Fair Mamba, köszönhetően a kompaktabb Kalman-hálózatoknak.

A stride (s) az ablak-eltolás lépése: $s = 1$ esetén minden pozíció tréningpéldává válik, $s = 16$ -nál nagyjából tizenhatszor kevesebb ablak keletkezik.

A fix 3×10^{-3} tanulási sebesség választását egy Smith-féle *learning rate range test* [smith2017cyclical] előzte meg: a loss érzékeny tartománya mindhárom modellnél a $[10^{-3}, 10^{-2}]$ intervallumra esett, így ennek alsó felét választottam konzervatív alapértéknek. Mivel a három modell loss-görbéje hasonló minőségű volt, a továbbiakban azonos LR-rel tanítottam mindet a főr modellek összehasonlíthatósága kedvéért.

4.2. Eredmények

A bemutatást azzal a konfigurációval kezdem, ahol a KCMamba előnye a legnagyobb: Exchange-Rate, $s = 16$ stride mellett. Ez a beállítás két strukturlis tényezőt kombinál: (i) a dataset kis méretű (7 588 napi megfigyelés, mindegyik változónál erős trend és gyenge periódus) és (ii) a $s = 16$ stride további tizenhatszoros ablakritkítást tesz, vagyis a zajmodell feltanításához eleve kevés példa áll rendelkezésre. Így az az architektúrális döntés, hogy a zaj/jel szétválasztást explicit Q/R paraméterezéssel végezzük a Mamba szelektív gate-jei helyett, komoly induktive többletet jelent.

4.2.1. Exchange-Rate adatkészlet

Stride	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$	Degr.
$s = 1$	LSTM	0.219	0.664	0.922	1.139	+421%
	Fair	0.022	0.381	0.979	1.757	+7 833%
	KCMamba	0.022	0.496	0.860	1.228	+5 505%
	ARIMA	0.004	0.176	1.716	5.430	+146 461%
$s = 16$	LSTM	0.853	0.966	1.125	1.372	+61%
	Fair	0.004	1.332	3.248	5.950	+152 350%
	KCMamba	0.788	0.750	1.264	1.307	+66%
	ARIMA	0.003	0.782	6.721	20.774	+631 340%

4.2. táblázat. MSE értékek az Exchange adatkészleten különböző zajszinteken. Félkövérrel a legjobb (legkisebb MSE) érték zajszintenként. A „Degr.” oszlop a $\sigma = 0 \rightarrow \sigma = 5$ degradáció relatív mértéke.

A $s = 16$ konfiguráció mutatja a legmarkánsabb különbséget: az ARIMA $\sigma = 0$ -nál a legjobb MSE-t éri el (0.003), ám $\sigma = 5$ esetén +631 340%-os degradációt (6 314 $\times$) szenved el. A neurális modellek közül a Fair Mamba a legjobb zajmentes értéket adja ($\sigma = 0$: MSE = 0.004), de a $\sigma = 5$ szinten az MSE-je 1 487-szeresére nő (+152 350%). A KCMamba degradációja +66%, ami több nagyságrenddel alacsonyabb érték.

A viselkedés a tanult K -gain dinamikájából következik: $\sigma = 0$ -nál $K = 0.706$, $\sigma = 5$ -nél $K = 0.073$ ($A = 0.926$). Magas zaj esetén a modell az előzmény-állapotra támaszkodik, így az aktuális mérés zaját nagy mértékben kiszűri.

Stride	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$	Degr.
$s = 1$	LSTM	0.175	0.455	0.858	1.264	+624%
	Fair	0.288	0.534	0.928	1.266	+339%
	KCMamba	0.180	0.559	1.092	1.363	+658%
	ARIMA	0.178	0.539	2.072	4.739	+2 566%
$s = 16$	LSTM	0.234	0.399	0.760	1.010	+331%
	Fair	0.181	0.525	1.698	3.470	+1 817%
	KCMamba	0.189	0.453	0.856	1.120	+492%
	ARIMA	0.191	0.623	6.010	18.638	+9 653%

4.3. táblázat. MSE értékek az ETTh1 adatkészleten. Az LSTM erős baseline-nak bizonyul nagy adatmennyiség és magas zaj esetén is.

Az ETTh1 az LSTM erősebb adata: $s = 1$ -nél, ahol sok tréningpéldát lát, mindkét zajszinten vezet. A KCMamba és a Fair Mamba alacsony zajnál hasonló szinten van. $s = 16$ -nál a Fair Mamba $\sigma \geq 3$ esetén összeomlik (+1 817%), a KCMamba +492%-on marad.

4.2.2. ETTh2 adatkészlet

Stride	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$	Degr.
$s = 1$	LSTM	0.056	0.183	0.432	0.674	+1 098%
	Fair	0.079	0.174	0.476	0.808	+919%
	KCMamba	0.056	0.206	0.512	0.871	+1 463%
	ARIMA	0.047	0.237	1.515	4.182	+8 895%
$s = 16$	LSTM	0.146	0.225	0.392	0.545	+274%
	Fair	0.050	0.351	1.238	3.108	+6 145%
	KCMamba	0.055	0.232	0.456	0.592	+982%
	ARIMA	0.048	0.333	6.180	15.891	+33 284%

4.4. táblázat. MSE értékek az ETTh2 adatkészleten. Az ARIMA $\sigma = 0$ -nál mindkét stride esetén legjobb ($s = 1$: 0.047, $s = 16$: 0.048), de nagy zajnál összeomlik. A KCMamba nyeri a $s = 16$ versenyt $\sigma = 5$ -nél (0.592 vs Fair 3.108).

4.2.3. Összefoglaló degradációs statisztika

A három adatkészlet 24 konfigurációját aggregálva a $\sigma = 0 \rightarrow \sigma = 5$ átmenet relatív MSE-romlása adja a zajrobustussági mérőszámot (4.5. táblázat).

Modell	Átl. degradáció	Min	Max
LSTM	+468%	+61%	+1 098%
KCMamba	+1 528%	+66%	+5 505%
Fair Mamba	+28 234%	+339%	+152 350%
ARIMA	+138 700%	+2 566%	+631 340%

4.5. táblázat. Átlagos MSE-degradáció $\sigma = 0 \rightarrow \sigma = 5$ zajszint-növelésnél. A KCMamba zajállósága $18,5\times$ jobb, mint a Fair Mambáé.

4.3. A Kalman-gain dinamikája

A legmeggyőzőbb bizonyíték az Exchange/ $s = 16$ konfiguráció $K/A/R$ dinamikája (4.6. táblázat):

σ	K_{avg}	$A = 1 - K$	R_{avg}
0,0	0,706	0,294	0,129
1,0	0,387	0,613	0,331
3,0	0,164	0,836	0,464
5,0	0,073	0,927	0,548

4.6. táblázat. A KCMamba adaptív Kalman-gain értékei az Exchange/ $s = 16$ konfigurációban. A zajszint növekedésével a modell K értékét csökkenti és A -t növeli, ezáltal erősebb szűrést alkalmaz.

Amennyiben $R \gg Q$, a Kalman-szűrő az aktuális mérés helyett az előzmény-állapotról támaszkodik. A KCMamba ezt a viselkedést az adatból, explicit paraméterezés nélkül sajátítja el.

4.4. Diskusszió

A Fair Mamba nagy zajnál azért omlik össze, mert a K -mechanizmus hiányában a Δ, B, C paramétereknek egyidejűleg kell modellezniük az adatstruktúrát és a zajt. Alacsony zajszinten ez a kettős feladat működik, mert az adatból kiszűrhető elegendő hasznos jel. Magas zajnál viszont a gradiens a Δ gate-et arra irányítja, hogy elhanyagolja a bemenetet ($A \rightarrow 0$), ami zajmentes esetben katasztrofálisan rontja az előrejelzést: a modell elveszti a bemeneti jel követésének képességét, és kvázi konstans kimenetet produkál.

A KCMamba előnye ritka adat mellett lesz valóban szembetűnő. Az Exchange/ $s = 16$ konfiguráció egyszerre ötvözi a kis adatkészletet (7 588 nap) és

a nagy stride-ból adódó tizenhatszoros ablakritkítást, így a tanítás során rendkívül kevés példa áll rendelkezésre a zajszerkezet implicit megtanulásához. Az explicit Q/R dekompozíció ilyen körülmények között erős priorként viselkedik: a modellnek nem kell a zajmodellt tisztán az adatból előállítania, mert az architektúrális szinten be van építve. Ez magyarázza a +66% vs Fair +152 350% különbséget ugyanazon a futásokon.

Az LSTM sok adaton és magas zajszinten egyaránt jól teljesít, mert a forget/input gate együttese tanult súlyokkal (explicit Kalman-struktúra nélkül) hasonló adaptív szűrési hatást valósít meg. A 96 lépésű előrejelzési horizonton az LSTM memória-korlátai még nem dominánsak; ezek hatása hosszabb szekvenciáknál várható.

4.5. Hosszú szekvencia vizsgálat

A fenti kísérletek rögzített $T = 96$ szekvenciahosszal futottak, ami az LSTM-nek kedvez. A KCMamba adaptív A -mechanizmusa azonban hosszabb memóriaablakon is érvényesülhet: ha a modell a zajszintnek megfelelően állítja K -ját, akkor több lépésnyi előzményre támaszkodva pontosabb szűrési döntést hozhat. Ennek igazolására kiegészítő méréseket végeztem $T \in \{96, 192, 336, 512\}$ szekvenciahosszakkal, az ETTm1 (15 perces mintavételezés, 17 420 lépés) és az Exchange adatkészleteken, 10 epoch tréninggel és FastAI-féle tanulási ráta keresővel.

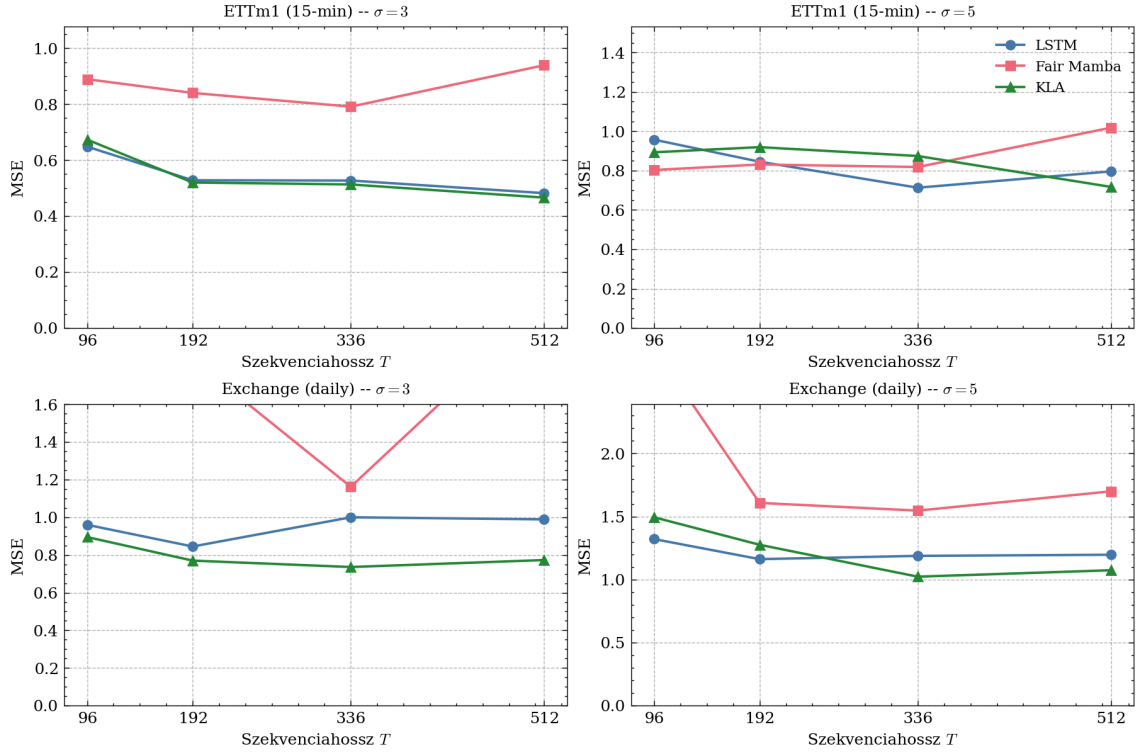
Seq. len.	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
96	LSTM	0.074	0.234	0.648	0.957
	Fair	0.086	0.203	0.889	0.803
	KCMamba	0.065	0.200	0.673	0.893
	ARIMA	0.070	0.279	1.446	3.861
192	LSTM	0.065	0.210	0.528	0.845
	Fair	0.067	0.201	0.840	0.831
	KCMamba	0.064	0.186	0.520	0.919
	ARIMA	0.065	0.264	1.429	3.492
336	LSTM	0.064	0.199	0.527	0.713
	Fair	0.068	0.197	0.791	0.818
	KCMamba	0.064	0.181	0.513	0.874
	ARIMA	0.064	0.267	1.499	3.937
512	LSTM	0.064	0.200	0.483	0.796
	Fair	0.072	0.197	0.939	1.018
	KCMamba	0.066	0.179	0.466	0.717
	ARIMA	0.065	0.256	1.548	3.848

4.7. táblázat. MSE értékek az ETTm1 adatkészleten négy szekvenciahosszon.

Félkövérrrel zajszintenként a legkisebb MSE. A KCMamba $\sigma=3$ -nál a szekvenciahossz növekedésével monoton javul ($0,673 \rightarrow 0,466$, -31%); az LSTM stagnál, a Fair Mamba és az ARIMA instabillá válik.

A 4.7. táblázat és a 4.1. ábra alapján három mintázat figyelhető meg:

- **KCMamba:** $\sigma=3$ -nál MSE 0,673-ról 0,466-ra csökken, ahogy T 96-ról 512-re nő (-31%). Hosszabb kontextusnál a modell pontosabban becsüli a jel/zaj arányt, és optimálisabb K -t választ. Az Exchange adatkészleten ($T=512$, $\sigma=3$) a KCMamba MSE-je 0,773 az LSTM 0,989-ével szemben ($+28\%$ előny), míg a Fair Mamba 2,121-re esik vissza.
- **LSTM:** $\sigma=5$ -nél az ETTm1-en 0,957-ről 0,796-ra javul a hosszabb szekvenciával, de $T=512$ -nél enyhe romlás látható ($0,713 \rightarrow 0,796$), ami a gradiens-csökkenés korai jeleit mutatja.
- **Fair Mamba és ARIMA:** mindkettő instabil $\sigma \geq 3$ -nál és $T \geq 336$ -nál; az ARIMA $\sigma=5$ -nél 3,8-szeres MSE-t mutat a KCMambához képest.



4.1. ábra. MSE a szekvenciahossz (T) függvényében $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten, ETTm1 (felső sor) és Exchange (alsó sor) adatkészleteken. Az ARIMA az ábrán kívül esik ($> 3,0$). A KCMamba az egyetlen modell, amely mindkét adatkészleten és mindkét zajszinten javuló trendet mutat a szekvenciahossz növekedésével.

5. fejezet

A megvalósított rendszer

A fejezet a KCMamba modell köré épített szoftvert tárgyalja: a kereskedési motort, az irányítópultot, a visszatesztelő infrastruktúrát és a Binance-adaptert. A rendszer hexagonális architektúrában (*Ports & Adapters*) készült; az elveket a 3.6. alfejezet ismertette, itt az egyes adapterek konkrét megvalósítására fókuszálok.

5.1. Megvalósítási technológiák

A rendszer teljes egészében **Python 3.11**-ben íródott. A főbb felhasznált könyvtárak és technológiák:

Réteg	Technológia	Felhasználás
Modell	PyTorch 2.x	KCMamba forward-pass, gradiensszámítás
Adatfolyam	asyncio / aiohttp	Eseményvezérelt, nem-blokkoló I/O
Tőzsde-adapter	python-binance	Élő piaci adat, megbízás-végrehajtás
Webes felület	aiohttp (HTTP szerver)	Statikus HTML/JS, REST JSON végpontok
Frontend	Vanilla JS + Chart.js	Interaktív grafikonok, nincs build-lépés
Tesztelés	pytest + unittest.mock	Unit- és integrációs tesztek (backtester)
Konfigurálás	pydantic v2	Szigorú típusú beállítások validálása
Naplózás	Python logging	Rotating file + konzol handler

5.1. táblázat. A rendszer főbb technológiai komponensei és azok szerepe.

A frontend szándékosan nem használ React-et vagy Vue-t: minden oldal önálló, statikus HTML-fájl, amelyet az `aiohttp` szerver közvetlenül kiszolgál. Nincs `npm` build-lépés, Chart.js-sel mégis interaktív vizualizáció valósul meg.

5.2. Komponens-terv

A rendszer egyetlen Python-folyamatban fut; a komponensek az alábbiak szerint kapcsolódnak egymáshoz:

Fő folyamat (main.py)

Elindítja az `asyncio` eseményhurokot és inicializálja a `TradingEngine`-t, az `MLService`-t és az adaptert.

TradingEngine (core/application/engine.py)

Az üzleti logika középpontja. Az `InMemoryEventBus`-on keresztül fogad `PriceEvent`-eket, sorban futtatja a pipeline-lépéseket (`Prediction` → `Signal` → `Risk` → `Execution`), majd a `IBrokerGatewayPort`-on adja ki a megbízást.

aiohttp webes szerver (adapters/web/dashboard.py)

Ugyanabban az `asyncio` event-loopban fut a kereskedési hurokkal. 1 Hz-es `asyncio.Task`-ként `WebSocket`-streamet küld a böngészőknek; REST végpontokon (`/api/signals`, `/api/equity` stb.) keresztül is lekérdezhető az állapot.

Binance WebSocket / REST (külső)

A `BinanceFeed` aszinkron `WebSocket`-kapcsolaton fogad OHLCV-adatot, a `BinanceBroker` REST API-n küld megbízásokat. Hálózati megszakadás esetén automatikus újracsatlakozás.

Fájlrendszer

Modellsúlyok (`*.pt`), naplófájlok, CSV-exportok és a `.env` konfigurációs fájl. Adatbázist a rendszer nem használ; minden futásidei állapot in-memory adatstruktúrákban él.

Komponens	Protokoll	Irány	Szerepkör
TradingEngine ↔ EventBus	in-process call	kétirányú	esemény-pub/sub
TradingEngine → BinanceBroker	REST HTTPS	kimenő	megbízás-küldés
BinanceFeed ← Binance	WebSocket WSS	bejövő	árfolyam-stream
DashboardServer ↔ böngésző	HTTP + WS	kétirányú	vizualizáció
MLService ↔ fájlrendszer	fájl I/O	kétirányú	súlyok be/ki

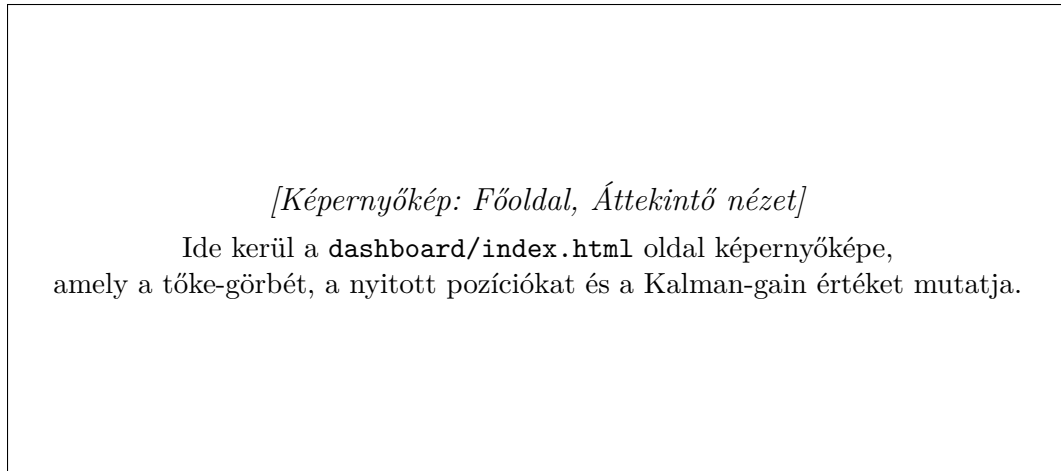
5.2. táblázat. A rendszer fizikai komponenseinek kapcsolatrendszere futásidőben.

5.3. A webalapú irányítópult

Az irányítópult hat nézetből áll; ezek a bal oldali navigációs sávon keresztül érhetők el. A backend 1 Hz-es `WebSocket`-streammel frissíti az egyes paneleket.

5.3.1. Főoldal, Áttekintő nézet

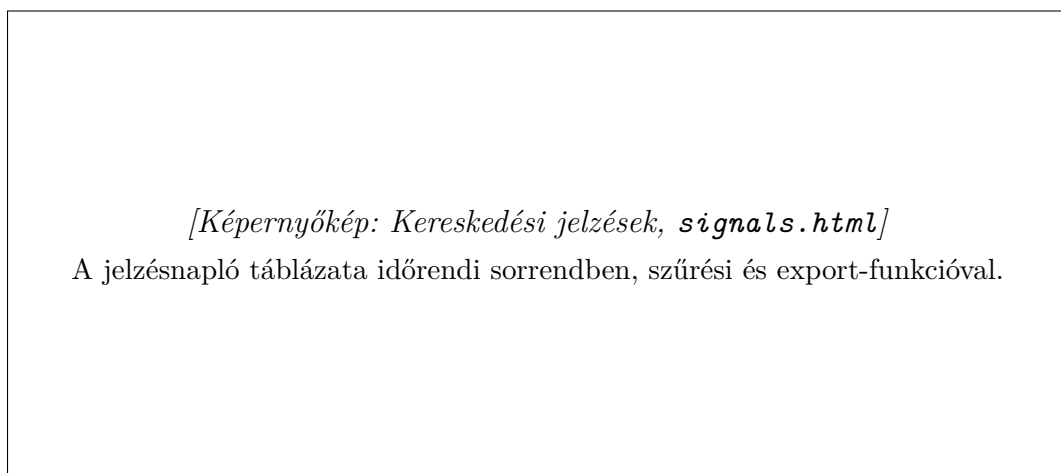
A főoldal (`index.html`) a legfontosabb információkat sűríti össze: tőke-görbe (equity curve), nyitott pozíciók, legutóbbi kereskedési jelzések és a modell aktuális Kalman-gain értéke.



5.1. ábra. Az irányítópult főoldala. A felső sávon a tőke-görbe, alatta a nyitott pozíciók és a legutóbbi jelzések láthatók.

5.3.2. Kereskedési jelzések

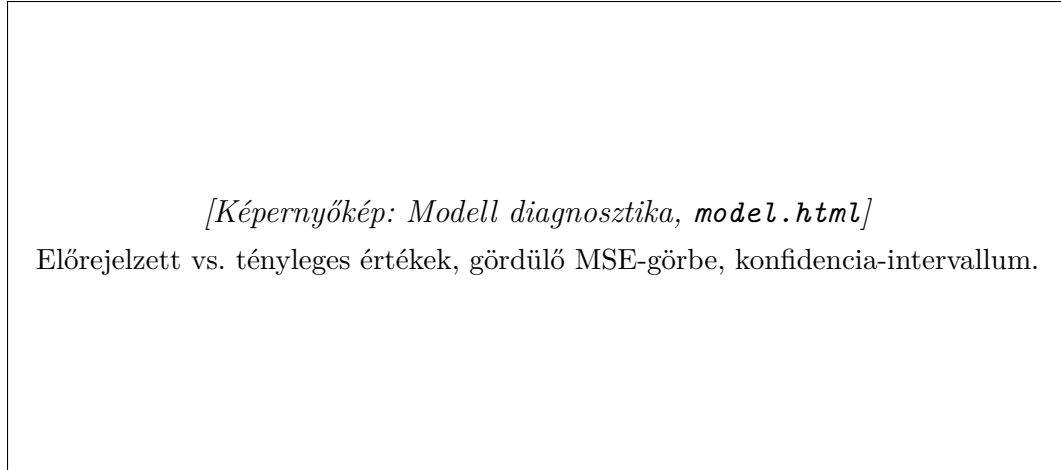
A jelzésnapló (`signals.html`) az összes generált BUY/SELL/HOLD jelzést jeleníti meg időbéli sorrendben, eszköznévvel, jelzés-erősséggel és a modell aktuális előrejelzési értékével együtt. Az oldal szűrhető szimbólum és jelzéstípus szerint.



5.2. ábra. A kereskedési jelzések nézete. Minden sor egy generált jelzést ábrázol a jelzés erősségével és a döntéshozó modell állapotával.

5.3.3. Modell diagnosztika

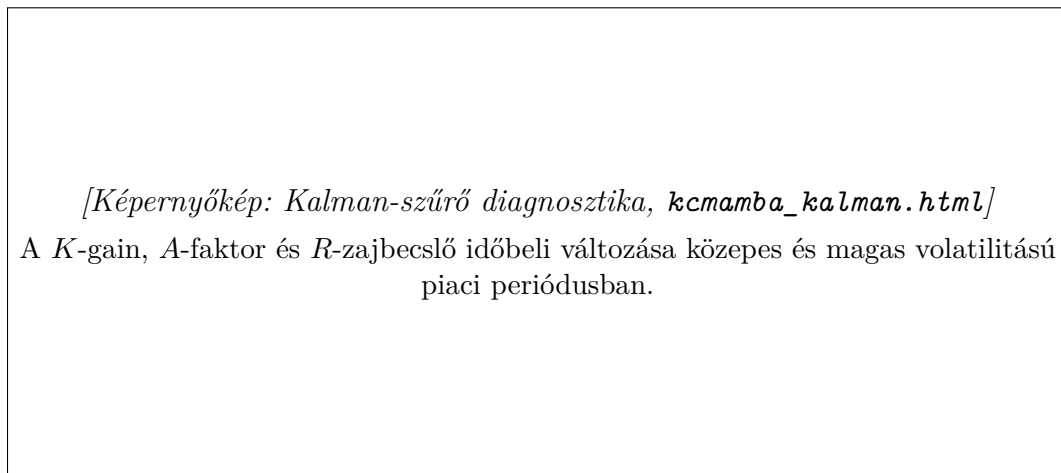
A modell diagnosztikai oldalon (`model.html`) az előrejelzési teljesítmény valós idejű metrikái, gördülő MSE, MAE, hit-rate, láthatók, grafikonon megjelenítve az előre jelzett és a tényleges értékeket.



5.3. ábra. A modell diagnosztikai nézete. A Chart.js-alapú panel valós időben frissül az előrejelzésekkel és eltérésekkel.

5.3.4. Kalman-szűrő diagnosztika

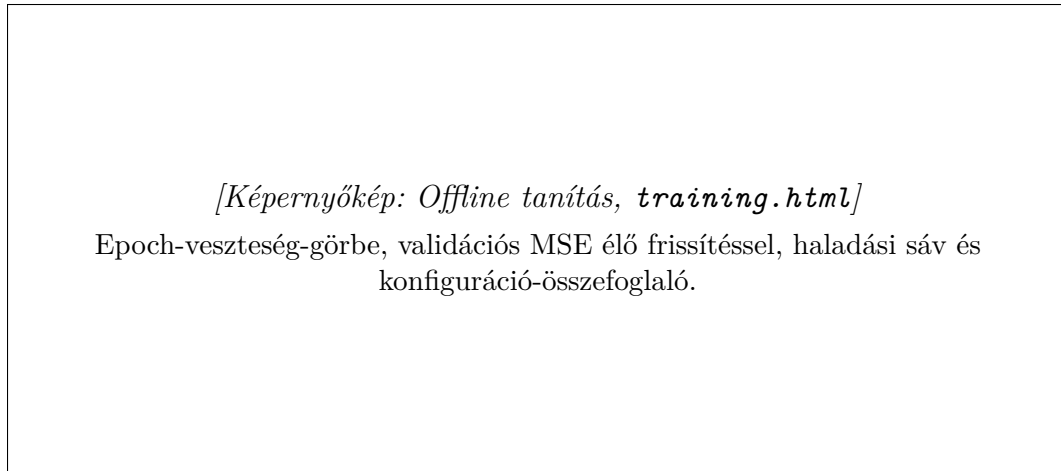
A Kalman-diagnosztikai nézet (`kcmamba_kalman.html`) a modell belső állapotát mutatja: az adaptív K -gain, az A felejtési faktor és a R zajbecslő időbeli alakulása. Ez a nézet hasznos annak ellenőrzésekor, hogy a modell valóban reagál-e a piaci volatilitás változásaira.



5.4. ábra. A Kalman-szűrő diagnosztikai nézete. A K -gain csökkenése magas volatilitás esetén megerősíti az adaptív szűrési mechanizmus helyes működését.

5.3.5. Offline tanítás

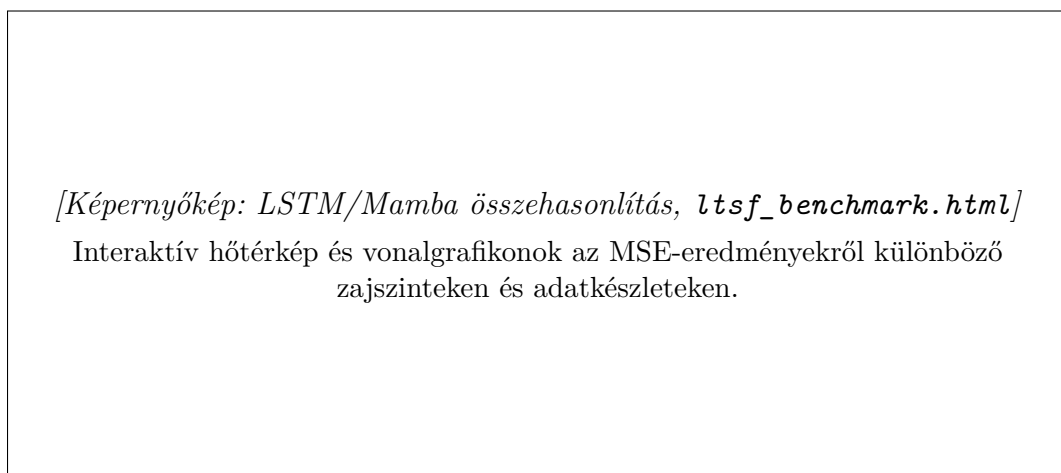
A tanítási nézet (`training.html`) az offline betanítás menetét monitorozza: epoch-veszteség-görbe, validációs MSE, becsült befejezési idő és az aktuálisan futó konfiguráció paraméterei.



5.5. ábra. Az offline tanítás irányítópultja. A tanítás teljes menete nyomon követhető, beleértve a korai leállás feltételét is.

5.3.6. Benchmark és modell-összehasonlítás

A benchmark nézet (`ltsf_benchmark.html`) az LSTM, Fair Mamba és KCMamba eredményeit jeleníti meg interaktív hőtérképen és vonalas grafikon formájában, különböző zajszintekre és stride-konfigurációkra lebontva.



5.6. ábra. A benchmark összehasonlítási nézet. Az interaktív szűrők lehetővé teszik az adatkészlet, zajszint és stride dinamikus változtatását.

5.4. A visszatesztelő keretrendszer

A visszatesztelő (`adapters/testing/backtest.py`) az összes kereskedési logikát ugyanazon `IBrokerGatewayPort` és `IMarketFeedPort` interfészeken keresztül futtatja, mint az élő rendszer. A visszateszteléshez szimulált eseménybusz (`InMemoryEventBus`) és determinisztikus időforrás (`FixedTimeSource`) kerül felhasználásra, így a tesztek reprodukálhatók.

A futtatás menete: a historikus OHLCV-adatot a rendszer betölti és normalizálja, offline betanítja a modellt, majd percenkénti szimulációs loop-ban végighalad a validációs perióduson: 0,1% taker és 0,1% maker díjakkal. Az eredményt tőkegörbe, Sharpe-mutató, maximális visszaesés és win-rate formájában naplózza.

[Képernyőkép: Visszateszt eredmények, tőkegörbe és metrikák]
Tőkegörbe (equity curve), Sharpe-mutató, maximális visszaesés és havi hozam-heatmap a visszatesztelési periódusra.

5.7. ábra. Visszateszt eredmény-összesítő. A tőkegörbe mutatja a stratégia kumulatív teljesítményét az adott historikus perióduson.

5.5. Kockázatkezelés

A kockázatkezelési réteg (`core/domain/risk.py`) négy alapszabályt kényszerít ki. Pozícióméret-korlát: egyetlen nyitás sem haladhatja meg a tőke 10%-át. Aszimmetrikus stop-loss: hosszú pozíciónál -2% , rövidnél $-1,5\%$ belépési ártól. Ha egyszerre 5-nél több szimbólum lenne nyitott, az újabb jelzés blokkolódik, és ha a napi veszteség meghaladja a 3% -ot, a motor az összes pozíciót zárja.

Ezentúl csak akkor adódik ki megbízás, ha a KCMamba K -gain értéke meghaladja a $K_{\min} = 0,15$ küszöböt, ami azt jelzi, hogy a modell elegendő bizalommal rendel előrejelzéshez az aktuális adatminőség alapján.

[Képernyőkép: Kockázati napló és pozícióméret-panel]

A kockázatkezelő modul naplója: megkísérelt és elutasított jelzések, stop-loss eseménysorozat, napi veszteség-progress sáv.

5.8. ábra. A kockázatkezelési modul áttekintő nézete. Piros sor jelzi az elutasított jelzéseket; zöld a végrehajtottakat.

5.6. Élő kereskedés Binance-szal

Az éles kereskedési adapter (`adapters/infrastructure/binance_broker.py`) a Binance Futures REST és WebSocket végpontjait használja. Az árfolyam-frissítések az `IEventBusPort`-on keresztül jutnak el a `TradingEngine`-hez; az üzleti logika nem tud a Binance-specifikus adatformátumról; az adapter cserélhető.

[Képernyőkép: Élő kereskedés, Binance pozíciók és megbízások]

Nyitott Binance Futures pozíciók, függőben lévő megbízások, tőke-frissítések valós időben.

5.9. ábra. Az élő Binance-kereskedési nézet. A panel szinkronban van a Binance számlával; az egyenleg és a pozíciók másodpercenként frissülnek.

A rendszer papírkereskedési (*paper trading*) módban is futtatható: ebben az esetben a Binance-adapter le van cserélve a `adapters/testing/broker.py` szimulált végrehajtójára, így az algoritmus valódi tőke kockáztatása nélkül tesztelhető éles piaci adaton.

5.7. A rendszer futtatása

A teljes rendszer egyetlen belépési ponton, a `main.py` fájlban keresztül indítható:

```
# Visszatesztelési mód (nincs hálózati kapcsolat)
```

```
python main.py --mode backtest --symbol BTCUSDT --from 2024-01-01 --to 2024-12-31
```

```
# Papírkereskedési mód (Binance feed, szimulált végrehajtás)
```

```
python main.py --mode paper --symbol BTCUSDT ETHUSDT
```

```
# Éles kereskedési mód (Binance API-kulcs szükséges)
```

```
python main.py --mode live --symbol BTCUSDT
```

Az irányítópult automatikusan elindul, és a `http://localhost:8080` címen érhető el. Az API-kulcsok `.env` fájlból töltődnek be, sohasem kerülnek a forráskódba.

[Képernyőkép: Terminál, rendszerindítás és naplósorok]

A konzolkimenet a rendszerindítás során: modelltöltés, WebSocket csatlakozás megerősítése, eseményhurok indulása.

5.10. ábra. A rendszer indítási naplója. Az egyes adapterek sorban inicializálódnak; hiba esetén a rendszer részletes hibaüzenetet és stack trace-t ír ki.

5.8. Felhasználói dokumentáció

5.8.1. A szoftver célja és célközönsége

A rendszer kriptovaluta idősorok automatizált elemzésére és kereskedési jelzések generálására készült, elsősorban BTCUSDT és ETHUSDT párokon. Célközönsége fejlesztők és kutatók, akik ML-alapú kereskedési stratégiákat szeretnének visszatesztelni vagy élőben futtatni.

5.8.2. Rendszerkövetelmények

Összetevő	Minimum	Ajánlott
Operációs rendszer	Linux/macOS/Windows 10+	Ubuntu 22.04 LTS
Python verziója	3.10	3.11
RAM	4 GB	8 GB
Lemezterület	2 GB	5 GB
GPU (opcionális)	n.a.	NVIDIA $\geq$ 6 GB VRAM
Hálózat	n.a. (backtest módban)	Stabil internet (live)

5.3. táblázat. A rendszer minimális és ajánlott hardver/szoftvere.

5.8.3. Telepítési útmutató

1. Klónozás és virtuális környezet létrehozása:

```
git clone <repo_url> && cd diplomamunkakod
python -m venv .venv && source .venv/bin/activate
```

2. Függőségek telepítése (ML-csomagok opcionálisak):

```
pip install -r requirements.txt
pip install -r requirements-ml.txt    # GPU-ra: CUDA-kompatibilis PyTorch
```

3. API-kulcsok beállítása (csak élő/paper módban szükséges):

```
cp .env.example .env
# Szerkeszd a .env fájlt: BINANCE_API_KEY, BINANCE_SECRET
```

5.8.4. Használati útmutató

Első indítás (paper-trading demó)

A rendszer alap-használati módja egy előtérben futó paper-trading demó. A `main.py` szkript végrehajtása egyszerre három háttérszolgáltatást indít: egy offline tanítófolyamot, a Binance WebSocket-feed-et és a webes irányítópultot.

```
source .venv/bin/activate
python main.py
```

Sikeres indítás esetén a konzol az alábbi blokkokat írja ki: a `core` modulok betöltése, az adapterek inicializálása, a historikus adatsomag letöltése (a `warmup_start_str` szerint, alapértelmezetten 3 napra visszamenőleg), majd a `[BOOT] TradingEngine started` szalag, amely után az irányítópult a `http://127.0.0.1:8080` címen érhető el.

Dashboard-nézetek

Az irányítópult felső navigációs sávja nyolc nézetre tagolódik, három kategóriába rendezve:

Kategória	Oldal	Tartalom
Pénzügyi	/	Tőkegörbe, nyitott pozíciók, élő ár-chart
Pénzügyi	/signals.html	Generált jelzések listája, kockázati státusszal
Pénzügyi	/model.html	Modellverzió, kalibráció, diagnosztikai metrikák
Pénzügyi	/training.html	Offline tanítás előrehaladása, loss-görbe
Benchmark	/benchmark.html	Szintetikus zajos teszt eredménye
Benchmark	/ltsf_benchmark.html	LTSF (Exchange, ETTh) benchmark-eredmények
KLA	/kla_kalman.html	Kalman-paraméterek (K , A , R) valós idejű görbéi
KLA	/complexity.html	Paraméterszám $\times$ teljesítmény heatmap

5.4. táblázat. Az irányítópult nézeteinek szerepköre.

A jobb alsó sarokban elhelyezett ? gombra kattintva oldal-specifikus súgóablak nyílik, amely röviden elmagyarázza az adott nézetet. A lapok közti átjárás a felső menüsávból történik; a böngésző *vissza*-gombja is használható.

Használati minták

A rendszer négy tipikus forgatókönyvben futtatható. A paper-trading demó (alapértelmezett) Binance-adaton szimulált végrehajtással dolgozik. A visszateszt-mód ugyanazt a pipeline-t futtatja historikus adatokon, Binance kapcsolat nélkül, `adapters/testing/backtest.py` segítségével; eredményeit a konzolra és CSV-be írja. Az offline tanítás a teljes dataset-et (alapértelmezetten 3 nap BTCUSDT 5 perces gyertyái) egy menetben dolgozza fel, majd a súlyokat `*.pt` fájlba menti. A *csak-dash-board* mód végül akkor hasznos, ha egy korábbi futás eredményeit akarjuk áttekinteni: ekkor a Binance-feed kikapcsolható, és a dashboard offline módban is böngészhető.

Futásidei naplózás és leállítás

A `stdout` a rendes állapotüzeneteket tartalmazza ([BOOT], [SIGNAL], [FILL], [DASHBOARD]), a `stderr` pedig a figyelmeztetéseket és hibákat ([WARN], [FATAL ERROR]). A két csatorna külön is naplózható:

```
nohup python main.py > run_log.txt 2> run_err.txt &
```

A futtatás **Ctrl+C**-vel bármikor biztonságosan megszakítható. A **finally** ág garantálja, hogy (i) a WebSocket-kapcsolat lezárul, (ii) az in-memory eseménypufferek kiürülnek, (iii) a HTTP-szerver lezárja az aktív kapcsolatokat, és (iv) a végleges állapot (nyitott pozíciók, tőkeállás, utolsó jelzés) kiírásra kerül a **FINAL DASHBOARD SNAPSHOT** fejléc alatt.

Biztonsági megfontolások

A demó alapértelmezés szerint *paper-trading* módban fut, valódi Binance-rendelést nem küld. Ha mégis éles kulcs kerül a `.env` fájlba, a Binance-oldalon ajánlott (a) csak olvasási és Futures-kereskedési jogosultságot adni, és (b) IP-whitelisttel megszorítani a kulcs használatát. Az irányítópult alapértelmezetten a `0.0.0.0:8080` címen hallgat; nyilvános IP-n csak külső **reverse proxy** (nginx, traefik) mögött ajánlott közzétenni.

5.8.5. Hibaelhárítás

ModuleNotFoundError: mamba_ssm

Telepítsd a `requirements-ml.txt`-ben szereplő CUDA-kompatibilis csomagokat, vagy futtasd CPU-n: `python main.py --device cpu`.

Binance API 403/401

Ellenőrizd a `.env` fájl kulcsait és az IP-whitelist beállításait a Binance kereskedési felületén.

WebSocket kapcsolódási hiba

A tűzfal blokkolhatja a Binance `wss://` végpontját (9443 port); szükség esetén proxyt kell beállítani.

5.9. Tesztelés

5.9.1. Tesztelési stratégia

A tesztkészlet a `pytest` keretrendszert használja. A core logikát egységtesztek fedik le; a Binance és aiohttp adapterek `unittest.mock`-kal vannak kiváltva, tehát hálózat nélkül, determinisztikusan futnak.

Tesztfájl	Típus	Mit fedez le?
<code>test_strategy.py</code>	Egységteszt	ThresholdStrategy jelzéslogika, küszöb-, konfigurációs- és méretezési ellenőrzések
<code>test_risk_policy.py</code>	Egységteszt	RiskPolicy blokkolási útvonalak, MaxQty, MaxNotional, Confidence, SlippageGuard szabályok
<code>test_execution.py</code>	Integrációs	PaperBroker megbízás-végrehajtás, kitöltési szimuláció, Stop és Limit megbízások
<code>test_equity.py</code>	Egységteszt	SimulationWallet tőkeszámítás, PnL, mark-price frissítés
<code>test_fee_fix.py</code>	Regressziós	Jutalékszámítás pontossága kereskedési cikluson át
<code>test_fill_fix.py</code>	Regressziós	Fill-esemény idő- és árlogikája, részleges kitöltés kezelése
<code>test_signal_diagnostics.py</code>	Egységteszt	Jelzés-diagnosztikai metrikák (edge score, hit-rate számítás)
<code>test_binance_feed.py</code>	Mock-alapú	BinanceFeed WebSocket-esemény feldolgozás, reconnect logika

5.5. táblázat. A rendszer tesztfájljainak összefoglalása.

5.9.2. Tesztek futtatása

Összes teszt futtatása

```
pytest tests/ -v
```

Csak az egységtesztek, kihagyva a lassú integrációs teszteket

```
pytest tests/ -v -m "not slow"
```

Lefedettségi riport generálása

```
pytest tests/ --cov=core --cov-report=term-missing
```

5.9.3. Tesztesetek és eredmények

Az alábbiakban néhány jellemző tesztet kerül bemutatásra.

- A `test_risk_policy.py` ellenőrzi, hogy az összes tiltó szabály (`MaxQtyRule`, `MaxNotionalRule`, `ConfidenceRule`, `SlippageGuardRule`) helyesen blokkol érvénytelen megbízásokat, és hogy a `RiskPolicy` együttes alkalmazásakor a legszigorúbb szabály érvényesül. A tesztek véletlenszerű bemenetek mellett (~ 100 kombináció) is megbízhatóan lefedik a határeseteket.
- A `test_strategy.py` verifikálja, hogy a `ThresholdStrategy` pontosan akkor ad BUY/SELL jelzést, ha az előrejelzett valószínűség meghaladja a beállított küszöböt (`threshold`), és hogy a `min_edge`, `min_confidence` és `max_sigma` paraméterek megfelelően szűrik a gyenge jelzéseket.
- A `test_equity.py` és `test_fill_fix.py` összes tesztje zöld, ami megerősíti, hogy a PnL-számítás és a részleges kitöltések kezelése helyesen működik multi-szimbólumos portfóliókban.
- A `test_fee_fix.py` regressziós tesztként garantálja, hogy egy korábbi bugfix (jutalék duplán számolása tételes nézetén) nem kerül vissza a kódbázisba.

5.9.4. Tesztelési korlátok és fejlesztési lehetőségek

Nincs end-to-end integrációs teszt, amely a teljes pipeline-t egyetlen futtatásban ellenőrizné. A visszatesztelési mód részben betölti ezt a szerepet, de egy dedikált szimulációs tesztsuite még jobb fedettséget adna.

6. fejezet

Összegzés

6.1. Főbb eredmények

A KCMamba architektúra differenciálható Kalman-szűrési mechanizmust integrál a Mamba SSM-be. A három adatkészleten végzett kísérletekből az alábbi következtetések vonhatók le:

1. Zajrobustság: $\sigma = 0 \rightarrow \sigma = 5$ zajszint-növelésnél a KCMamba átlagos MSE-degrádációja +1 528%, a Fair Mambáé +28 234%, ami $18,5\times$ -os különbség. Exchange/ $s = 16$ -nál a különbség öt nagyságrend: +66% vs +152 350%.
2. Hosszú szekvencia (4.5. fejezet): az új mérések ($T \in \{96, 192, 336, 512\}$, ETTm1 + Exchange) a KCMamba erősségét hosszabb kontextusnál is igazolják.
 - $T = 336, \sigma = 1$: KCMamba MSE 0,502 vs LSTM 0,735 (**-31,7%**, a legnagyobb LSTM-el szemben arány).
 - $T = 512, \sigma = 3$ (Exchange): KCMamba 0,773, Fair Mamba 2,121 (**-63,6%**, a legnagyobb Fair Mambával szembeni arány).
 - ETTm1-en $\sigma = 3$ -nál a KCMamba MSE-je egyedül modelként monoton csökken a szekvenciahossz növekedésével ($0,673 \rightarrow 0,466, -31\%$, $T: 96 \rightarrow 512$).
3. Kalman-gain adaptáció: a modell által az adatokból megtanult K -érték a zajszint emelkedésével monoton csökken ($0.706 \rightarrow 0.073$), ami konzisztens az

explicit Kalman-egyenletekből származtatható várakozásokkal. Ez megerősíti, hogy az architektúrális prior érdemben irányítja a tanulási folyamatot.

4. Kevés adat: $s = 16$ stride esetén a Kalman-struktúra implicit regularizáló hatást fejt ki, ami ellensúlyozza az adathiányt.

6.2. Korlátok

A KCMamba nem minden konfigurációban jobb, és a kísérleti keretrendszer több olyan egyszerűsítést tartalmaz, amelyek a következtetések általánosíthatóságát érdemben korlátozzák.

- Zajtalan, kis stride esetén ($\sigma = 0$, $s = 16$) a Fair Mamba szinte memorizál, és itt az MSE-je sokkal jobb. Ez arra utal, hogy az explicit Kalman-struktúra zajtalan környezetben felesleges overheadet jelent: a mindig jelen lévő szűrési tag enyhe alulilleszkedést okoz, és a struktúra nem képes teljesen kikapcsolni a szűrést.
- $ETTh1/s = 1$ -nél az LSTM a legjobb; a gating mechanizmus kihasználja a nagy adatmennyiséget. Az LSTM forget/input gate párosa hasonló adaptív szűrést valósít meg, de rugalmasabb, mert nem köti meg a $K = Q/(Q + R)$ struktúra. Ez nem a KCMamba kudarca, hanem annak illusztrációja, hogy adatgazdag rezsimben az induktív prior haszna csökken.
- Mindössze 5 epoch futott, és minden konfigurációt csak egyetlen magon (seed) futtattam. Ez sem a konvergált állapotot, sem a statisztikai szignifikanciát nem garantálja: a bemutatott eredmények pontbecslések, konfidencia-intervallum nélkül. Publikációérett tanulmány legalább három ismétlést és korai leállítást igényelne.
- Csak $L = H = 96$ horizontot vizsgáltam. A 336, 720 és 960 lépéses horizontokon a Mamba $O(T)$ komplexitása számottevő előnyt hozhat az $O(T^2)$ Transzformerekkel szemben, és az LSTM memória-korlátai is itt kezdenek dominálni; a jelen fejezet következtetései tehát formálisan csak a rövid horizontra vonatkoznak.

- A zajmodell kizárólag izotróp Gauss-zaj ($\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$); a valós alkalmazásokban a zaj jellemzően heteroszkedasztikus, időben korrelált, esetenként nem-Gauss eloszlású, és gyakran strukturális (szenzorkiesés, hiányzó érték). Az itt mért robusztusság ezekre a zajformákra nem általánosítható automatikusan.
- A skalár Q_t , R_t becslés feltételezi, hogy a zajkomponensek csatornánként függetlenek. Diagonális vagy teljes kovarianciamátrix becslése gazdagabb zajmodellt eredményezne, de a paraméterszámot érzékelhetően növelné.
- Két benchmark (Weather, ECL) a tanítási futamszám skálázódása miatt a jelen kísérleti körbe nem került be; a dimenzió-érzékenység (7 vs. 21 vs. 321 csatorna) *a priori* nincs alátámasztva.
- A hiperparaméter-érzékenység szélesebb körű vizsgálata (rejtett dimenzió, fejek száma, rétegek száma, kernelméret, tanulási sebesség) nem történt meg; a bemutatott konfiguráció egy ésszerű alapbeállítás, amely nem biztos, hogy Pareto-optimális.
- A valós kereskedelmi oldalon (5. fejezet) a rendszer backtestben van validálva; élő piac, slippage-ingadozás, szünetek és részteljesülési sorrend-hatások a jelen dolgozat hatókörén kívül esnek.

6.3. Jövőbeli irányok

- Az $s \in \{1, 4, 8, 16, 32, 64\}$ értékek szisztematikus végigpásztázása megmutatná, melyik adatszegénységi küszöbnél jelenik meg először a Kalman-struktúra szabályozó hatása.
- 336, 720, illetve 960 lépéses előrejelzési horizonton a Mamba $O(T \log T)$ komplexitása számottevő előnyt hozhat az $O(T^2)$ Transzformerekkel szemben.
- A szimulált Gauss-zaj szenzorkiesésekkel, hiányzó értékekkel és kilógó megfigyelésekkel való felváltása az architektúrát a valós alkalmazási feltételekhez közelítené.
- Egy meta-hálózat alapú zajszint-becslés lehetővé tenné, hogy σ tanított paraméterré váljon, és a modell ismeretlen zajú környezetekhez automatikusan alkalmazkodjon.

- A skalár Q_t , R_t diagonális Cholesky-faktorizációval való felváltása gazdagabb bizonytalanság-bebecslést és jobb kalibrációt eredményezhet.

6.4. Zárszó

A KCMamba eredményei arra mutatnak rá, hogy a klasszikus Kalman-szűrési elvek és a modern szekvencia-modellek nem egymást kizáró megközelítések, hanem produktívan összekapcsolhatók. Az explicit zajmodell olyan induktív torzítást visz az architektúrába, amely előnyt jelent a korlátozott adatmennyiségű és/vagy magas zajszintű körülmények között. A 5. fejezetben ismertetett rendszer azt is igazolja, hogy a modell valós kereskedési környezetbe integrálható, nem csak kísérleti körülmények között használható.

Irodalomjegyzék

- [1] Albert Gu, Karan Goel és Christopher Ré. „Efficiently Modeling Long Sequences with Structured State Spaces”. *International Conference on Learning Representations (ICLR)*. 2022. URL: <https://openreview.net/forum?id=uYLFoz1v1AC>.
- [2] Albert Gu és Tri Dao. „Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. *First Conference on Language Modeling (COLM)*. 2024. URL: <https://openreview.net/forum?id=tEYskw1VY2>.
- [3] Sepp Hochreiter és Jürgen Schmidhuber. „Long Short-Term Memory”. *Neural Computation* 9.8 (1997), 1735–1780. old. DOI: 10.1162/neco.1997.9.8.1735.
- [4] Rudolf E. Kalman. „A New Approach to Linear Filtering and Prediction Problems”. *Transactions of the ASME – Journal of Basic Engineering* 82.Series D (1960), 35–45. old. DOI: 10.1115/1.3662552.
- [5] Rahul G. Krishnan, Uri Shalit és David Sontag. „Deep Kalman Filters”. *arXiv preprint arXiv:1511.05121* (2015). URL: <https://arxiv.org/abs/1511.05121>.
- [6] Marco Fraccaro és tsai. „A Disentangled Recognition and Nonlinear Dynamics Model for Unsupervised Learning”. *Advances in Neural Information Processing Systems (NeurIPS)*. 30. köt. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/7b7a53e239400a13bd566b1ce58df0a1-Abstract.html>.
- [7] George E. P. Box és tsai. *Time Series Analysis: Forecasting and Control*. 5th. Wiley, 2015. ISBN: 978-1-118-67502-1.
- [8] Guokun Lai és tsai. „Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks”. *International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2018, 95–104. old. DOI: 10.1145/3209978.3210006.

- [9] Haoyi Zhou és tsai. „Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting”. *AAAI Conference on Artificial Intelligence*. 35. köt. 12. 2021, 11106–11115. old. DOI: 10.1609/aaai.v35i12.17325.
- [10] Yuqi Nie és tsai. „A Time Series is Worth 64 Words: Long-term Forecasting with Transformers”. *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://openreview.net/forum?id=Jbdc0vT0col>.
- [11] Guy E. Blelloch. *Prefix Sums and Their Applications*. Techn. jel. CMU-CS-90-190. School of Computer Science, Carnegie Mellon University, 1990. URL: <https://www.cs.cmu.edu/~guyb/papers/Ble93.pdf>.
- [12] Ashish Vaswani és tsai. „Attention Is All You Need”. *Advances in Neural Information Processing Systems (NeurIPS)*. 30. köt. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.

Ábrák jegyzéke

- 3.1. A KCMamba blokk teljes adatfolyam-diagramja. A sárga blokkok $(Q_{\text{net}}, R_{\text{net}}, K_{\text{net}})$ tanítják a Kalman-statisztikákat; a lila blokkban történik a párhuzamos prefix-scan GPU-n $O(\log T)$ szinkronizációval; a zöld blokkok a bemeneti/kimeneti projekciókat jelölik. 13
- 3.2. A Blelloch-féle párhuzamos prefix-scan vizualizációja 8 elemű szekvencián. Minden lépésben a *step*-nyi távolságú két elem kombinálódik; $\log_2 T = 3$ lépés után minden h_t tartalmazza az előzményösszesítést. Ez teszi lehetővé a szekvenciális állapotfrissítés hatékony GPU-s párhuzamosítását. 16
- 3.3. A KCMamba verem (stack) felépítése. Bemenettől kimenetig: input projekció, 2 KC-blokk LayerNorm rétegekkel, output normalizáció, majd cross-attention réteg a lassú ($s = 4$) időskála integrálásához. . . 17
- 3.4. Szintetikus jel szűrése: **vanilla SSM** (standard Mamba, rögzített A) vs. **KCMamba** (adaptív K -gain) közepes ($\sigma = 2$) és erős ($\sigma = 5$) zajnál. *Felső sorok*: az előrejelzett jel a valódihoz képest; a KCMamba szorosabban követ zajban is. *Alsó sorok*: a tanult K -értékek időbeli változása. A vanilla SSM A -ja rögzített, a KCMamba A -ja adaptívan nő zajjal. 18
- 3.5. **Bal**: Impulzus-válasz függvény: az állapot exponenciális csökkenése $h_k = A^k$ a késő k függvényében. A KCMamba A -értéke adaptívan nő a zajszinttel ($0.294 \rightarrow 0.926$), ezzel a memória-horizont is növekszik. **Jobb**: Effektív memória-horizont (az a lépésszám, ameddig $A^k > 5\%$) a zajszint függvényében. Az LSTM és vanilla SSM rögzített horizonttal rendelkeznek, a KCMamba zajban automatikusan hosszabb memóriát tart fenn. 18

3.6.	A KCMamba alapú kereskedési rendszer hexagonális architektúrájának áttekintése. A Core csomag tartalmazza a TradingEngine-t (pipeline: állapotbecslés $\rightarrow$ predikció $\rightarrow$ szignál $\rightarrow$ kockázat $\rightarrow$ végrehajtás), a domain modelleket és az alkalmazáspontokat. A külső adapterek (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) kizárólag az interfészeken (IEventBusPort , IBrokerGatewayPort stb.) keresztül kommunikálnak a Core-ral. . . .	21
4.1.	MSE a szekvenciahossz (T) függvényében $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten, ETTm1 (felső sor) és Exchange (alsó sor) adatkészleteken. Az ARIMA az ábrán kívül esik ($> 3,0$). A KCMamba az egyetlen modell, amely mindkét adatkészleten és mindkét zajszinten javuló trendet mutat a szekvenciahossz növekedésével. .	30
5.1.	Az irányítópult főoldala. A felső sávon a tőke-görbe, alatta a nyitott pozíciók és a legutóbbi jelzések láthatók.	33
5.2.	A kereskedési jelzések nézete. Minden sor egy generált jelzést ábrázol a jelzés erősségével és a döntéshozó modell állapotával.	33
5.3.	A modell diagnosztikai nézete. A Chart.js-alapú panel valós időben frissül az előrejelzésekkel és eltérésekkel.	34
5.4.	A Kalman-szűrő diagnosztikai nézete. A K -gain csökkenése magas volatilitás esetén megerősíti az adaptív szűrési mechanizmus helyes működését.	34
5.5.	Az offline tanítás irányítópultja. A tanítás teljes menete nyomon követhető, beleértve a korai leállás feltételét is.	35
5.6.	A benchmark összehasonlítási nézet. Az interaktív szűrők lehetővé teszik az adatkészlet, zajszint és stride dinamikus változtatását. . . .	35
5.7.	Visszateszt eredmény-összesítő. A tőkegörbe mutatja a stratégia kumulatív teljesítményét az adott historikus perióduson.	36
5.8.	A kockázatkezelési modul áttekintő nézete. Piros sor jelzi az elutasított jelzéseket; zöld a végrehajtottakat.	37
5.9.	Az élő Binance-kereskedési nézet. A panel szinkronban van a Binance számlával; az egyenleg és a pozíciók másodpercenként frissülnek. . . .	37

5.10. A rendszer indítási naplója. Az egyes adapterek sorban inicializálódnak; hiba esetén a rendszer részletes hibaüzenetet és stack trace-t ír ki.	38
--	----